eJPT Study Guide: Module 1.2 Footprinting and Scanning

Source: INE eJPT course, module "1.2 Footprinting & Scanning" (Introduction To Network Mapping). Order of blocks follows the order of the course screenshots.

Table of Contents

1. Module at a Glance 4

2. Big Picture Map 6

3. Concept-by-Concept Guide 7

3.1 Network Mapping in the Penetration Testing Flow 7
3.2 Network Protocols and Packets 9
3.3 The OSI Model 10
3.4 The Network Layer: IP, ICMP and DHCP 11
3.5 The IPv4 Header 13
3.6 The Transport Layer, TCP, the Handshake and TCP Flags 15
3.7 TCP Port Ranges 16
3.8 TCP versus UDP 18
3.9 Listing Local Ports with netstat 18
3.10 Host Discovery Techniques 20
3.11 Ping Sweeps and fping 21
3.12 Nmap Host Discovery: -sn versus -Pn 22
3.13 Ping Sweep with IP Packets: --send-ip 24
3.14 Scanning a Target List: -iL 25
3.15 TCP SYN Ping: -PS 27
3.16 TCP ACK Ping: -PA 28
3.17 ICMP Echo Ping: -PE 29
3.18 Verbose Ping Scans and Several Probes: -v, -T4 29
3.19 Full-Port, Service and OS Scan: -sS -sV -O --osscan-guess -p- 31
3.20 Scan Techniques and Port Specification 33
3.21 Service Version Detection and Intensity 35
3.22 The Nmap Scripting Engine and Default Scripts: -sC 36
3.23 Finding Scripts and Reading Their Help 38
3.24 Running Chosen Scripts: --script 39
3.25 Firewall Evasion: Packet Fragmentation (-f, --mtu) 41
3.26 ACK Scan to Map Firewall Rules: -sA 43
3.27 Payload Length and Decoys (--data-length, -D) 45
3.28 Other Spoofing Options: -S, -e, -g, --proxies, --data 46
3.29 Scan Timing and Performance 47
3.30 Scanning a Subnet and --host-timeout 48
3.31 Nmap Output and Verbosity 50

4. Chained Scenarios 52

Scenario A: The host that ignores ping 52
Scenario B: Services on odd ports 52
Scenario C: Firewall in the way, then the next subnet 53

5. Decision Guide and Exam Strategy 54

5.1 If I see X, I try Y 54
5.2 Enumeration checklist for this module 54
5.3 Common traps 55
5.4 How to take notes in the lab 55
5.5 Turning findings into answers 55

6. Practice 56

6.1 Lab-Style Tasks 56
6.2 Exam-Style Questions 60
6.3 Answer Key and Explanations 60

7. Recap Card and Flashcards 62

7.1 Recap Card 62

7.2 Flashcards 62
8. Glossary 64
9. Coverage Report 66
9.1 Page to Section Map 66
9.2 [UNCLEAR] Items 67
9.3 Items to Double Check 67
9.4 Confirmation 67

1. Module at a Glance

Module name: 1.2 Footprinting and Scanning (Introduction to Network Mapping).

Exam domains fed:

- Assessment Methodologies (25 percent): the main one. Host discovery, port scanning, service and OS fingerprinting, and the penetration testing flow all live here.
- Host and Network Auditing (25 percent): NSE scripts and service checks are used to audit what each host exposes.
- Host and Network Penetration Testing (35 percent): every attack starts from the port and version list you build in this module.

What the exam expects from this module: In the live lab you must find which hosts are alive, which TCP and UDP ports are open on each, which service and version runs on each port, and what the operating system is. Questions are answered only from what you find. A missed port or a missed host means a missed question and, later, a missed pivot.

The 10 things to remember most:

1. Order of work: find live hosts -> scan ports -> detect versions and OS -> run scripts.
2. A default Nmap scan checks only the 1000 most common TCP ports. Lab services hide on odd ports (55413, 41288, 10000 in these notes), so use `-p-`.
3. "Host seems down" does not mean the host is down. A firewall may block pings. Try `-Pn` or other probe types.
4. `-sn` means discovery only (no port scan). `-Pn` means skip discovery and scan anyway.
5. `-sS` (SYN scan) needs root and is the default for root. `-sT` (connect scan) works without root.
6. Port states: open, closed, filtered. Filtered means a firewall hides the answer.
7. `-sV` finds service versions, `-O` guesses the OS, `-sC` runs the default NSE scripts.
8. NSE scripts live in `/usr/share/nmap/scripts/`. Find them with `ls | grep`, read them with `--script-help`.
9. Firewall tricks: `-f` with `--mtu` (multiple of 8), `-D` decoys, `--data-length`, and `-sA` to map firewall rules.
10. Protocol facts: TCP handshake is SYN, SYN/ACK, ACK. ICMP echo is type 8, reply is type 0. IP protocol numbers: ICMP 1, TCP 6, UDP 17.

Skill checklist (I can ...):

- I can explain the five phases of the penetration testing methodology and say which phase scanning belongs to.
- I can name the 7 OSI layers and place IP, ICMP, TCP, UDP, HTTP, and Ethernet on the right layer.
- I can read an IPv4 header and a TCP header and name the key fields.

- I can draw the TCP three-way handshake and list the TCP flags.
- I can list local listening ports with `netstat -antp`.
- I can sweep a subnet with `fping` and with `nmap -sn`.
- I can choose between `-Pn`, `-PS`, `-PA`, and `-PE` when a host does not answer pings.
- I can scan all 65535 TCP ports, detect versions, and guess the OS.
- I can find, read, and run an NSE script for a given service.
- I can fragment packets, add decoys, and run an ACK scan to learn about a firewall.
- I can speed up or slow down a scan and stop a slow host with `--host-timeout`.

> **REMEMBER:** Everything in this guide is for authorized labs and legal practice only. Scan only systems you own or have written permission to test.

2. Big Picture Map

Scanning is the first active step of a penetration test. Its output (hosts, ports, versions) feeds every later step. In the exam you repeat it on each new subnet you reach through pivoting.

```
1 Scoping (which IP ranges are allowed)
  |
2 Passive recon (OSINT, DNS, search engines, Google dorks)
  |
3 ACTIVE RECON <==== THIS MODULE
  |   host discovery -> port scan -> version and OS -> NSE scripts
  v
4 Enumeration (SMB shares, FTP, HTTP, users)
  |
5 Vulnerability assessment
  |
6 Exploitation (initial access)
  |
7 Post-exploitation and privilege escalation
  |
8 Pivoting (new subnet found -> go back to step 3 on that subnet)
  |
9 Reporting
```

Step	Role of this module	Output you keep
Scoping	Know the target range before any scan	List of allowed IPs
Recon	Active half: touch the target	Live host list
Enumeration	Ports and versions decide what to enumerate	Port and service table
Exploitation	Version numbers map to exploits	Version strings
Pivoting	Same scans, run from the compromised host	New subnet host list

3. Concept-by-Concept Guide

3.1 Network Mapping in the Penetration Testing Flow

In one line: Network mapping is the active work of finding which hosts exist, which doors (ports) they have open, and what lives behind each door.

Theory in simple words. A penetration test follows a fixed methodology. Your notes show five phases:

- Information Gathering. Passive (OSINT, which means Open Source Intelligence, public information) and active (network mapping, host discovery, port scanning, service detection and OS detection).
- Enumeration. Service enumeration, user enumeration, share enumeration.
- Exploitation (initial access). Vulnerability analysis and threat modeling (vulnerability analysis, vulnerability identification), then exploitation (developing or modifying exploits, service exploitation).
- Post-Exploitation. Local enumeration, privilege escalation, credential access, persistence, defense evasion, lateral movement.
- Reporting. Report writing and recommendations.

Active information gathering is the phase where the tester talks directly to the target to collect data and find possible weak points. It goes beyond passive reconnaissance, where you collect information without touching the target. Active work includes scanning, probing, and direct interaction with network services.

Analogy: planning to visit a building. Passive recon is reading its public floor plan and website. Active recon is walking up and trying the doors. The building owner can see you try the doors.

The notes split information gathering like this:

```
Information Gathering
  +-- Passive
  |     +-- Domain and IP analysis
  |     +-- Social media
  |     +-- Search engines
  |     +-- DNS recon
  |     +-- Google dorks
  +-- Active
        +-- Scanning
        |     +-- Network mapping / host discovery
        |     +-- Port scanning
        |     +-- Service and OS fingerprinting
        +-- Enumeration
```

Course topics in this module and where each is explained:

Topic in the course overview	Guide blocks
Networking fundamentals	3.2 to 3.9
Host discovery with Nmap	3.10 to 3.18

Topic in the course overview	Guide blocks
Port scanning with Nmap	3.19, 3.20
Host fingerprinting with Nmap	3.19, 3.21
Nmap Scripting Engine (NSE)	3.22 to 3.24
Firewall detection and evasion	3.25 to 3.28
Scan timing and performance	3.29, 3.30
Output and verbosity	3.18, 3.31

Learning objectives (from the notes), as skills: understand why network mapping matters; know the OSI model, TCP, and UDP; map a network and find hosts with Nmap; find open ports and services; fingerprint OS and services; detect and evade firewalls; speed scans up or down.

> **ADDED CONTEXT:** The March 2026 exam update added target scoping and passive versus active reconnaissance. Scoping means you confirm the exact IP ranges or domains you may test before you scan. In the lab, the briefing gives the range. Scan only that range.

How it works (the mapping routine):

1. Confirm the scope (the allowed range).
2. Find your own IP and subnet (see 3.4).
3. Run host discovery to list live hosts (3.10 to 3.18).
4. Run port scans on each live host (3.19, 3.20).
5. Run version and OS detection on the open ports (3.19, 3.21).
6. Run NSE scripts for interesting services (3.22 to 3.24).
7. Write every host, port, service, and version into your notes as you go.

Worked example. The lab brief says the target range is 10.10.10.0/24.

- Step 1: `nmap -sn 10.10.10.0/24` lists hosts 10.10.10.1, 10.10.10.5, and 10.10.10.7 as up (see 3.12).
- Step 2: `nmap -sS -sV -p- 10.10.10.5` shows 22/tcp ssh and 55413/tcp ftp (see 3.19).
- Step 3: you note "10.10.10.5: ssh, ftp on odd port 55413" and move to enumeration.

Lab question angle. "How many hosts are alive in the subnet?" or "Which port is the FTP service on?" Answer with a discovery scan and a full-port scan, not from memory.

Why it works and how defenders stop it. Hosts must answer network traffic to be useful, so they leak their presence. Defenders block ICMP, filter ports with firewalls, and watch for scan patterns with an IDS (intrusion detection system).

Memory aid. Who, Where, What. Who is alive (host discovery). Where are the open doors (port scan). What runs behind each door (version and OS).

3.2 Network Protocols and Packets

In one line: A protocol is an agreed set of rules for talking over a network, and a packet is the unit of data that carries the talk.

Theory in simple words. Hosts communicate through network protocols. Protocols let systems with different hardware and software understand each other. Many protocols exist, and each service uses its own for its own job (web uses HTTP, mail uses SMTP). The conversation between hosts is carried by packets.

A packet has two parts:

- **Header:** control data with a protocol-specific layout. It tells the receiver how to read the packet and how to handle the whole exchange.
- **Payload:** the real information. It can be part of an email or a slice of a file being downloaded.



Physically, a packet is a stream of bits sent as electric signals (or light or radio) on the medium, such as Ethernet or Wi-Fi. The receiver turns the signals back into zeros and ones.

Analogy: a packet is a letter in an envelope. The envelope has the addresses (header). The letter inside is the payload.

How it works:

1. An application creates data (for example a web request).
2. Each layer adds its own header around the data.
3. The bits travel over the medium.
4. The receiver reads the headers layer by layer and hands the payload to the right application.

Worked example. You open a web page from 10.10.10.5. Your browser data (payload) gets a TCP header (ports), then an IP header (addresses), then an Ethernet header (MAC addresses). Wireshark shows each header as a separate expandable row.

Lab question angle. Scan questions rarely ask for header theory directly. You need it to understand why Nmap changes header fields (fragmentation in 3.25, flags in 3.20) and to read packet captures.

Why it works and how defenders stop it. Headers are public, standard formats, so anyone can craft packets. Firewalls and IDS inspect header fields (source, destination, flags) to block or alert.

Memory aid. Header = envelope, payload = letter.

3.3 The OSI Model

In one line: OSI (Open Systems Interconnection) is a 7-layer reference model that splits network communication into ordered jobs.

Theory in simple words. The ISO (International Organization for Standardization) developed the model so different systems and devices can work together. Each layer has one function. The model is a guide for understanding and designing protocols. It is not a strict blueprint for every real network.

Analogy: sending a parcel. Packing (layer 7), translating the label language (6), tracking the delivery session (5), choosing reliable or fast shipping (4), planning the route (3), loading onto the truck (2), the road itself (1).

#	Layer	Function (from notes)	Examples (from notes)
7	Application	Network services for users and apps	HTTP, FTP, IRC, SSH, DNS
6	Presentation	Format translation, encryption, compression	SSL/TLS, JPEG, GIF, SSH, IMAP
5	Session	Sessions, sync, dialog control, tokens	APIs, NetBIOS, RPC
4	Transport	End-to-end delivery and flow control	TCP, UDP
3	Network	Logical addressing and routing	IP, ICMP, IPSec
2	Data Link	Media access, framing, error checks	Ethernet, PPP, switches
1	Physical	The physical connection	USB, cables, coax, fiber, hubs

Notes on the table:

- Layer 6 text: data is translated "to ensure it is presented in a readable format". Layer 5 is "interhost communication". Layer 3 is "logical addressing". Layer 2 is "physical addressing" (MAC addresses).
- The notes list SSH on both layer 7 and layer 6, and IMAP on layer 6. Many references put IMAP on layer 7. Treat this as a loose model. The exam does not depend on it. Double check if a question asks.

How it works (what the layers mean for a pentester):

1. Layer 2: ARP finds hosts on the local network (3.10).
2. Layer 3: IP and ICMP carry pings and routes (3.4, 3.11).
3. Layer 4: TCP and UDP ports are what a port scan tests (3.6 to 3.8).
4. Layer 7: services such as HTTP, FTP, SMB are what version detection and NSE scripts talk to (3.19, 3.22).

Worked example. Match Nmap actions to layers. An ARP ping works at layer 2. `nmap -PE` sends ICMP at layer 3. `nmap -sS` sends TCP SYN at layer 4. `nmap -sV` talks to the application at layer 7.

Lab question angle. "Which layer does TCP work at?" -> 4. "Which layer handles routing?" -> 3. Short recall questions are possible in the Assessment Methodologies domain.

Why it works and how defenders stop it. Each layer can be attacked or defended on its own. Defenders place controls at several layers (switch port security at 2, firewalls at 3 and 4, web firewalls at 7).

Memory aid. Layers 1 to 7: "Please Do Not Throw Sausage Pizza Away" (Physical, Data link, Network, Transport, Session, Presentation, Application).

3.4 The Network Layer: IP, ICMP and DHCP

In one line: Layer 3 gives every host a logical address (IP) and moves packets between networks.

Theory in simple words. The Network Layer handles logical addressing, routing, and forwarding of packets between devices on different networks. It finds a good path from source to destination. It hides the physical network so many networks act as one internetwork.

Internet Protocol (IP) is the central protocol here. It does logical addressing, routing, and fragmentation and reassembly of packets. It gives a standard way to identify and locate hosts.

IPv4 vs IPv6:

Property	IPv4	IPv6
Address size	32 bits	128 bits
Written as	four octets with dots	hexadecimal groups
Example	192.168.0.1	2001:0db8:85a3:0000:0000:8a2e:0370:7334
Why it exists	most widely used	fixes IPv4 address exhaustion

IPv4 has a finite address space. That shortage led to IPv6.

IP functions (from the notes):

- **Logical addressing:** IP addresses are logical addresses given to network interfaces. They identify each device. They are hierarchical: organized by network classes, subnets, and CIDR (Classless Inter-Domain Routing) notation.
- **Packet structure:** IP wraps data in packets. Each has a header and a payload. The header carries source and destination IP, version, TTL (time-to-live), and protocol type (details in 3.5).
- **Fragmentation and reassembly:** when a packet crosses a network with a smaller MTU (Maximum Transmission Unit, the biggest packet that link carries), IP splits it into fragments. The receiving host puts the fragments back together.
- **Address types:** unicast (one to one), broadcast (one to all in a subnet), multicast (one to a chosen group).
- **Subnetting:** splitting a large network into smaller ones for efficiency and security.

ICMP (Internet Control Message Protocol) is tied closely to IP. It reports errors and supports diagnostics. Examples: echo request and echo reply (the `ping` tool), traceroute, and various error messages.

DHCP (Dynamic Host Configuration Protocol) gives IP addresses automatically to devices, so nobody configures each host by hand.

Reserved IPv4 ranges (from the notes):

- 0.0.0.0 to 0.255.255.255 is "this" network.
- 127.0.0.0 to 127.255.255.255 is the local host (your own computer).
- Details of special-use addresses are in RFC 5735 (RFC 5735 was replaced by RFC 6890; verify).

> **ADDED CONTEXT:** Private ranges you will meet in labs: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. The address 169.254.0.0/16 is link-local (used when no DHCP answer comes). In the netstat output in 3.9, 169.254.169.254 is the cloud metadata address.
>
> **ADDED CONTEXT:** CIDR size guide for scan targets.

CIDR	Addresses	Typical use
/32	1	one host
/24	256	one lab subnet, e.g. 10.10.10.0/24
/16	65,536	large; scan slowly
/8	16,777,216	far too big for one scan

How it works:

1. A host sends a packet with source IP and destination IP.
2. If the destination is on another network, the host sends it to the default gateway (the router).
3. Each router looks at the destination IP and forwards the packet.
4. If a link has a smaller MTU, the packet is fragmented. The destination reassembles it.

Hands-on (ADDED CONTEXT): find your own IP and subnet before scanning.

Tool: `ip` (from the iproute2 package, preinstalled on Kali). Purpose: show your interfaces and routes so you know which subnet to scan.

```
ip -br a
ip route
```

Part	Meaning
ip	the iproute2 command
-br	brief, one line per interface
a	short for address
route	show the routing table

```
lo       UNKNOWN   127.0.0.1/8 ::1/128
eth0     UP        10.10.10.2/24
```

```
default via 10.10.10.1 dev eth0
10.10.10.0/24 dev eth0 proto kernel scope link src 10.10.10.2
```

Reading it: `lo` is the loopback (127.0.0.1, the local host range above). `eth0` has 10.10.10.2 with a /24 mask, so your subnet is 10.10.10.0/24 and the router is 10.10.10.1. Mistake to avoid: scanning your own IP and counting it as a target.

Worked example. `ip -br a` shows eth0 at 10.10.10.2/24. You decide to scan 10.10.10.0/24 and ignore .2 (yourself) and .1 (router). After you compromise a host with a second interface, run `ip a` there to find the hidden subnet.

Lab question angle. "What is the IPv4 address length?" -> 32 bits. "Which protocol is used by ping?" -> ICMP. "What is the network range of the internal subnet?" -> read it from the compromised host's interfaces and routes.

Why it works and how defenders stop it. IP trusts the source address in the packet, and ICMP gives away which hosts are alive. Defenders filter ICMP, apply ingress filtering, and segment networks.

Memory aid. IP = where (address), ICMP = what went wrong (messages), DHCP = who gets which address.

3.5 The IPv4 Header

In one line: The IPv4 header is the control block at the front of every IP packet, and Nmap and defenders both read and change its fields.

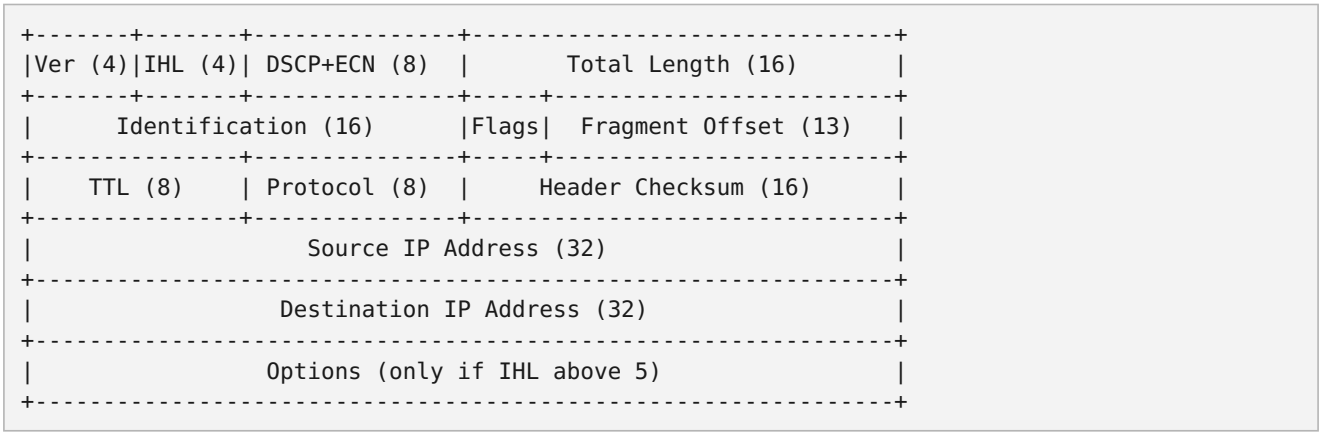
Theory in simple words. The IP header holds binary values that routers and hosts use to forward packets. Key fields named in the notes: source address, destination address, TTL, ToS, and protocol.

Field	Size	Purpose
Version	4 bits	IP version. For IPv4 the value is 4
Header Length (IHL)	4 bits	Header size in 32-bit words
Type of Service	8 bits	Priority. Holds DSCP and ECN
Total Length	16 bits	Whole packet size. Max 65,535 bytes
Identification	16 bits	Groups fragments of one packet
Flags	3 bits	Fragmentation control
Fragment Offset	13 bits	Position of this fragment
Time-to-Live (TTL)	8 bits	Max hops left. Minus 1 per router
Protocol	8 bits	Payload type: 1 ICMP, 6 TCP, 17 UDP
Header Checksum	16 bits	Error check of the header
Source IP	32 bits	Sender address
Destination IP	32 bits	Receiver address
Options	variable	Only if IHL is above 5

Details from the notes:

- **Header Length:** minimum value 5 = 20-byte header. Maximum 15 = 60-byte header.
- **Type of Service:** originally set quality of service. It contains DSCP (Differentiated Services Code Point) and ECN (Explicit Congestion Notification) for priority and congestion control. The layout shows DSCP as 6 bits and ECN as 2 bits.
- **Flags (3 bits):** bit 0 Reserved (always 0). Bit 1 DF, Don't Fragment (1 means do not fragment). Bit 2 MF, More Fragments (1 means more fragments follow).
- **TTL:** when it reaches 0 the packet is discarded. This is how loops end.
- **Protocol:** tells IP which higher-layer protocol receives the payload.

Layout (each row is 32 bits wide):



How it works:

1. The sender fills the header. TTL starts at a value set by the OS.
2. Every router lowers TTL by one and forwards the packet.
3. At TTL 0 a router drops the packet and sends an ICMP "time exceeded" message back. Traceroute uses this.
4. The receiver reads Protocol and passes the payload to ICMP, TCP, or UDP.

> **ADDED CONTEXT:** Typical starting TTL values: Linux and macOS 64, Windows 128, many network devices 255 (verify, they can be changed). A reply with TTL 127 or 128 suggests Windows. A reply with TTL 63 or 64 suggests Linux. It is a hint, not proof.

Worked example. You ping a host and the reply shows `ttl=127`. The start value is probably 128 (Windows) and the packet crossed one router. You note "likely Windows, 1 hop behind a router", then confirm with `nmap -O` (see 3.19).

Lab question angle. "What protocol number identifies TCP in the IP header?" -> 6. "What is the minimum IP header length?" -> 20 bytes. "Which flag stops fragmentation?" -> DF.

Why it works and how defenders stop it. Header fields are trusted by default, which allows spoofing and fragmentation tricks (3.25, 3.28). Defenders reassemble fragments before inspection and drop spoofed sources.

Memory aid. Protocol numbers: 1 = ICMP, 6 = TCP, 17 = UDP ("1, 6, 17"). Header sizes: 5 words minimum = 20 bytes.

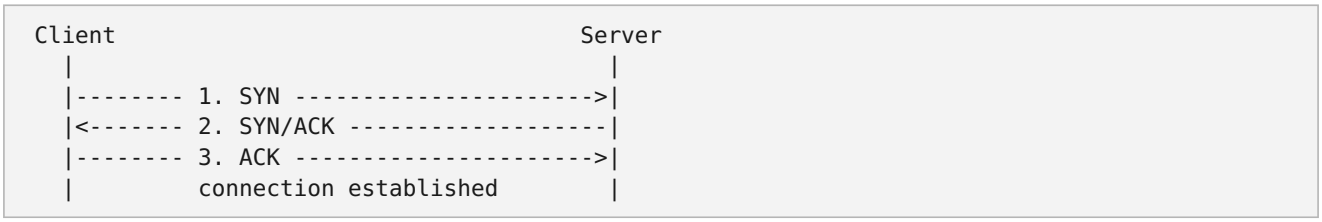
3.6 The Transport Layer, TCP, the Handshake and TCP Flags

In one line: The Transport Layer (layer 4) delivers data between two applications, and TCP does it reliably, in order, using a connection.

Theory in simple words. The Transport Layer provides end-to-end communication. It does error detection, flow control (do not send faster than the receiver can take), and segmentation (cutting data into smaller units). TCP (Transmission Control Protocol) is connection-oriented and reliable. It makes sure data sent by one application arrives accurately and in the right order at the other application.

Analogy: a phone call. You dial, the other side picks up, you both confirm you can hear, and only then do you talk. UDP (3.8) is more like dropping a postcard in the mailbox.

The three-way handshake is how TCP opens a connection. Three messages go between the client and the server.



TCP control flags (from the notes):

- Opening request: SYN set, ACK clear, FIN clear. "SYN" starts a connection request.
- Response: SYN set, ACK set, FIN clear. SYN acknowledges the request, ACK acknowledges the received data.
- FIN is the flag for ending a connection.

> **ADDED CONTEXT:** The other flags. RST resets (refuses or kills) a connection. PSH asks the receiver to pass data to the application now. URG marks urgent data. CWR and ECE are for congestion control. Nmap uses these flags to probe ports (3.20).

TCP header (from the notes):

```
+-------------------------------+-------------------------------+
|       Source Port (16)        |     Destination Port (16)     |
+-------------------------------+-------------------------------+
|                     Sequence Number (32)                      |
+---------------------------------------------------------------+
|                  Acknowledgment Number (32)                   |
+-------+-----+-----------------+-------------------------------+
|Off (4)| Rsv |    Flags (9)    |       Window Size (16)        |
+-------+-----+-----------------+-------------------------------+
|         Checksum (16)         |      Urgent Pointer (16)      |
+-------------------------------+-------------------------------+
|                   Options (if Off above 5)                    |
+---------------------------------------------------------------+
```

The notes show the flags CWR, ECE, URG, ACK, PSH, RST, SYN, FIN. The Flags (9) box also holds one more bit, NS, which the notes leave out (verify). In the diagram, Off is the data offset, and Rsv is the reserved bits. Flag bits in order: NS CWR ECE URG ACK PSH RST SYN FIN. Data offset works like IP header length: it says where the data starts.

How it works (how this explains port states):

1. Nmap sends SYN to a port.
2. SYN/ACK back -> the port is open (something listens).
3. RST back -> the port is closed (nothing listens, host is up).
4. Nothing back, or an ICMP error -> filtered (a firewall drops it).

Hands-on (ADDED CONTEXT): watch a handshake. Tool: Wireshark (packet analyzer, on Kali, run as `wireshark`). Apply this display filter to see only first SYN packets:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Part by part: `tcp.flags.syn == 1` means the SYN bit is set. `tcp.flags.ack == 0` means ACK is not set. Together they match only connection requests, which is what a SYN scan sends.

Worked example. You run a scan against 10.10.10.5 while Wireshark listens. Port 22 shows SYN -> SYN/ACK -> RST (Nmap resets instead of finishing). Port 23 shows SYN -> RST/ACK. You conclude: 22 open, 23 closed.

Lab question angle. "Which flag does the server set in the second step of the handshake?" -> SYN and ACK. "What does Nmap receive from a closed port?" -> RST.

Why it works and how defenders stop it. Every TCP service must answer SYN to be reachable, so the answer reveals the port state. Defenders drop unwanted SYN packets (filtered ports) and limit connection rates.

Memory aid. "SYN, SYN-ACK, ACK": Hello, Hello back, Okay.

3.7 TCP Port Ranges

In one line: A port is a 16-bit number that picks the service on a host, from 0 to 65,535.

Theory in simple words. An IP address finds the building. The port finds the apartment. Port numbers are 16-bit unsigned integers, so the maximum is 65,535. TCP uses ports to tell services and applications apart on one device.

Registered ports (1024 to 49151) are registered for specific services. IANA (Internet Assigned Numbers Authority) usually assigns them to software vendors. They are not standardized like the well-known ports, but often used for known services. Examples from the notes:

- 3389: Remote Desktop Protocol (RDP)
- 3306: MySQL database
- 8080: HTTP alternative port
- 27017: MongoDB database

> **ADDED CONTEXT:** The three ranges are well-known (0 to 1023), registered (1024 to 49151), and dynamic or private (49152 to 65535). The notes only show the registered range. Ports you should know on sight:

Port	Service	Port	Service
21	FTP	139	NetBIOS session (netbios-ssn)
22	SSH	143	IMAP
23	Telnet	443	HTTPS
25	SMTP	445	SMB (microsoft-ds)
53	DNS	3306	MySQL
80	HTTP	3389	RDP
110	POP3	8080	HTTP alternative
135	MSRPC	11211	memcached (verify)

> **ADDED CONTEXT:** In the scans in 3.12 and 3.26, Windows hosts show unknown ports 49152 to 49155 or 49154 and 49155. These are in the dynamic range. On Windows they are usually RPC (remote procedure call) endpoints. Ports 135, 139, 445, and 3389 together point to Windows.

How it works:

1. A server program binds to a port and listens.
2. A client picks a random high source port and connects to the server port.
3. The pair of (IP, port) on both sides identifies one conversation.

Worked example. A scan shows 3306/tcp open. You read it as MySQL, but you confirm with `-sV` (3.19), because admins can move services. In the notes, FTP ran on 55413 and Tomcat on 10000.

Lab question angle. "Which service runs on port 27017?" -> MongoDB. "What is the maximum port number?" -> 65535. "Which port holds the web admin service?" -> find it with a full-port scan and version detection.

Why it works and how defenders stop it. Default ports make services easy to find. Defenders move services, close unused ports, and filter at the firewall. Moving a port is not a real defense, since `-p-` and `-sV` find it anyway.

Memory aid. Max port = 65535 = 2 to the power 16, minus 1. "3389 RDP, 3306 MySQL, 27017 Mongo."

3.8 TCP versus UDP

In one line: TCP is a reliable connection, UDP is a fast, no-guarantee message.

Feature	UDP	TCP
Connection	Connectionless	3-way handshake
Reliability	No guaranteed delivery	Guaranteed, ordered, retransmits
Header size	Smaller, low overhead	Larger
Used for	VoIP, streaming, gaming	HTTP, email
Examples	DNS, DHCP, SNMP, SIP, games	HTTP, FTP, Telnet, SMTP, HTTPS

Theory in simple words. UDP (User Datagram Protocol) sends packets without setting up a connection and without checking they arrive. That makes it faster and lighter. TCP checks everything and resends lost data. Analogy: TCP is registered mail with a signature, UDP is a flyer through the door.

Why it matters for scanning.

- TCP scans get clear answers (SYN/ACK or RST).
- UDP often gives no answer when a port is open, so UDP scans are slow and less certain. Nmap's `-sU` option does it (3.20).
- Services that use UDP show up only if you scan UDP: DNS (53), DHCP (67/68), SNMP (161).

Worked example. A TCP scan of 10.10.10.5 shows no DNS port. You run a UDP scan of port 53 and find it open. The host runs DNS over UDP only.

Lab question angle. "Which protocol is connectionless?" -> UDP. "Which protocol uses a three-way handshake?" -> TCP. "SNMP uses which transport?" -> UDP.

Why it works and how defenders stop it. UDP has no handshake, so it is easy to spoof and to abuse. Defenders filter UDP services that are not needed.

Memory aid. TCP = Telephone (call set up first). UDP = postcard.

3.9 Listing Local Ports with netstat

In one line: `netstat` shows the network connections and listening ports on the machine you are logged in to.

Theory in simple words. Every open port on a host belongs to a program that is listening. `netstat` (from the net-tools package; on newer systems `ss` replaces it, verify) lists them. This is the view from inside. Nmap gives the view from outside. After you compromise a host, it shows you which services run there, who is connected, and which ports are open only on localhost.

Tool first seen here: `netstat`. Purpose: list sockets with their state and the owning program.

```
netstat -antp
```

Part	Meaning
netstat	the tool
-a	all sockets: listening and connected
-n	numeric: show IPs and ports, do not resolve names
-t	TCP only
-p	show PID and program name (root needed to see all)

Sample output (from the notes; spacing and two column titles shortened to fit):

```
Active Internet connections (servers and established)
Proto RQ  SQ Local Address   Foreign Address     State       PID/Prog
tcp    0   0 127.0.0.54:53   0.0.0.0:*           LISTEN      -
tcp    0   0 127.0.0.53:53   0.0.0.0:*           LISTEN      -
tcp    0   0 127.0.0.1:20241 0.0.0.0:*           LISTEN      -
tcp    0   0 0.0.0.0:22      0.0.0.0:*           LISTEN      -
tcp    0   0 10.0.0.66:50538 169.254.169.254:80  ESTABLISHED -
tcp    0 448 10.0.0.66:22    49.47.135.188:51882 ESTABLISHED -
tcp6   0   0 :::4444         :::*                LISTEN      825/python3
tcp6   0   0 :::80           :::*                LISTEN      -
tcp6   0   0 :::22           :::*                LISTEN      -
tcp6   0   0 ::1:3350        :::*                LISTEN      -
tcp6   0   0 :::3389         :::*                LISTEN      -
```

The tool also printed: "Not all processes could be identified, non-owned process info will not be shown, you would have to be root to see it all." That is why most PID columns show a dash.

How to read it, line by line:

- Columns: Proto (tcp for IPv4, tcp6 for IPv6), RQ and SQ (Recv-Q and Send-Q, bytes waiting), Local Address (your side), Foreign Address (other side), State, PID/Program.
- `127.0.0.54:53`, `127.0.0.53:53`: listening on loopback only. Local DNS helper (a stub resolver; verify). Not reachable from outside.
- `127.0.0.1:20241`: listening on loopback only. Not reachable remotely.
- `0.0.0.0:22`: SSH listens on every IPv4 interface. Reachable from outside.
- `10.0.0.66:50538` to `169.254.169.254:80` ESTABLISHED: this host is talking to the address 169.254.169.254, the cloud metadata service (added knowledge).
- `10.0.0.66:22` from `49.47.135.188:51882` ESTABLISHED: someone is connected by SSH right now.
- `:::4444` with `825/python3`: process 825, a Python program, listens on port 4444 on all addresses.
- `:::80`, `:::22`, `:::3389`: web, SSH, and RDP listen on all addresses.
- `::1:3350`: loopback only (IPv6 localhost).

Useful variations:

- `sudo netstat -antp` shows every PID and program.
- `netstat -antup` also lists UDP sockets (`-u`).
- `netstat -lntp` shows only listening TCP sockets (`-l`).
- `ss -tulpn` is the modern equivalent (verify flags on your system).

Common mistakes: running without root and then wondering why program names are missing. Mixing up Local and Foreign address.

Worked example. After you gain a shell on 10.10.10.5, you run `netstat -antp`. You see `127.0.0.1:3306 LISTEN`. MySQL is not visible from outside, but it exists. You note it for pivoting or port forwarding later.

Lab question angle. "Which program listens on port 4444?" -> read the PID/Program column (needs root). "Is the database reachable from the network?" -> check if it listens on 127.0.0.1 or 0.0.0.0.

Why it works and how defenders stop it. The OS keeps a table of sockets. Defenders audit it, close unneeded listeners, and bind internal services to loopback.

Memory aid. a-n-t-p: All, Numeric, TCP, Program. "Antp" like "ant".

3.10 Host Discovery Techniques

In one line: Host discovery finds out which IP addresses in a range belong to live machines.

Theory in simple words. Before you scan ports, you list who is alive. Scanning every port on 254 dead addresses wastes time. The notes show three methods:

- **Ping sweeps (ICMP echo requests):** send ICMP Echo Request to a range of IPs. Replies mark live hosts. Quick and common.
- **ARP scanning:** send ARP (Address Resolution Protocol) requests to find hosts on the local network. It works only inside the same broadcast domain (the same local network segment).
- **TCP SYN ping (half-open):** send TCP SYN to one port (often 80). A SYN/ACK reply means the host is alive. The notes call this stealthier than an ICMP ping.

Analogy: knocking on every door in a street versus asking the street's mailman who lives where (ARP).

How it works:

1. Pick a method that matches the network position. Same subnet -> ARP. Across routers -> ICMP or TCP probes.
2. Send probes to the range.
3. Mark every responder as live.
4. If a probe type gets no answers, try another (3.12 to 3.18).

Worked example. Your Kali box is 10.10.10.2/24. ARP finds neighbors on that subnet even if they block ICMP. Hosts behind a router (another subnet) cannot be found by ARP. You use ping sweeps or SYN pings for those.

Lab question angle. "How many hosts are alive in this subnet?" Pick a discovery method. If a host is known to exist but does not respond, change the probe type before saying it is down.

Why it works and how defenders stop it. ARP cannot be blocked on a local network because it is needed to talk. ICMP and SYN pings can be filtered. Defenders drop ICMP echo and rate-limit probes.

Memory aid. Local = ARP. Remote = ICMP or SYN ping.

3.11 Ping Sweeps and fping

In one line: A ping sweep sends ICMP echo requests to many addresses, and `fping` does it in one command.

Theory in simple words. A ping sweep sends one or more crafted ICMP packets of Type 8 (Echo Request) to a host. If the host replies with an ICMP packet of Type 0 (Echo Reply), the host is alive. ICMP messages carry a type and a code:

Message	Type	Code
Echo Request	8	0
Echo Reply	0	0

Tool introduced: `fping`. A small program like `ping` that probes many hosts at once, in parallel. Install on Kali if missing with `sudo apt install fping` (verify).

Command from the notes:

```
fping -a -g 10.10.23.0/24
```

Part	Meaning
fping	the tool
-a	print only hosts that are alive
-g	generate a target list from a range or CIDR
10.10.23.0/24	the 254 usable hosts 10.10.23.1 to 10.10.23.254

The notes show the command typed with no output yet. Typical output:

```
10.10.23.1
10.10.23.2
```

Reading it: each line is an alive host. Other lines can appear on stderr, such as "ICMP Host Unreachable from ...". They are noise from routers about dead addresses.

Variations:

- `fping -a -g 10.10.23.0/24 2>/dev/null` hides the stderr noise.

- `fping -a -g 10.10.23.1 10.10.23.50` sweeps from a start IP to an end IP.
- `fping -a -g 10.10.23.0/24 -q` hides per-host status lines.
- `fping -a -g 10.10.23.0/24 2>/dev/null > alive.txt` saves the live list for later scans.

> **ADDED CONTEXT:** Plain `ping -c 2 10.10.23.2` tests one host. `-c 2` means send 2 packets then stop (without it Linux pings forever). Sample output line: `64 bytes from 10.10.23.2: icmp_seq=1 ttl=127 time=0.8 ms`. The `ttl` value hints at the OS (see 3.5).

Common mistakes: concluding a host is down because it did not answer ICMP. Windows hosts block echo by default (verify), so a sweep misses them (see 3.12).

Worked example. Goal: list live hosts in 10.10.23.0/24. Run `fping -a -g 10.10.23.0/24 2>/dev/null`. You see .1 and .2. You write both down and move on to port scans. Because ICMP can be blocked, you also run `nmap -sn` (3.12) as a cross-check.

Lab question angle. "How many hosts respond on the subnet?" -> count lines. "Which ICMP type is an echo reply?" -> 0.

Why it works and how defenders stop it. Most hosts answer echo by default. Defenders block or rate-limit inbound ICMP echo at the firewall.

Memory aid. 8 asks, 0 answers. fping: -a alive, -g generate.

3.12 Nmap Host Discovery: `-sn` versus `-Pn`

In one line: `-sn` only checks which hosts are up. `-Pn` skips that check and scans the ports anyway.

Tool introduced: Nmap (Network Mapper). Nmap is the main scanner of the eJPT. It finds hosts, open ports, service versions, and operating systems, and it can run scripts (NSE). On Kali it is installed at `/usr/bin/nmap`. Many scan types send raw packets, so run Nmap as root (`sudo nmap ...`). The notes use Nmap 7.70 and 7.91. Check yours with `nmap --version`.

> **REMEMBER:** Output blocks marked "from the notes" are copied from your screenshots (IPs and times kept as shown). Blocks marked "illustrative" are typical output written for practice.

Theory in simple words. By default Nmap runs host discovery first, then scans ports only on hosts that answered. If a firewall blocks the discovery probes, Nmap decides the host is down and never scans it. A live host is lost.

Default discovery probes when run as root against a remote network (verified in the Nmap reference): ICMP echo request, TCP SYN to port 443, TCP ACK to port 80, and ICMP timestamp request. This equals `-PE -PS443 -PA80 -PP`. For targets on your own Ethernet segment, Nmap uses ARP instead. A non-root user gets a TCP connect to ports 80 and 443 only.

Analogy: a courier who leaves if nobody answers the doorbell (`-sn`) versus one who checks every window anyway (`-Pn`).

Command 1 (from the notes): skip discovery and scan.

```
nmap -Pn 10.4.31.111
```

Part	Meaning
nmap	the scanner
-Pn	no ping: treat the host as up, do not run host discovery
10.4.31.111	the target IP

Output (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 12:00 IST
Stats: 0:00:01 elapsed; 0 hosts completed (1 up), 1 undergoing SYN
Stealth Scan
SYN Stealth Scan Timing: About 11.25% done; ETC: 12:01 (0:00:16 remaining)
Nmap scan report for 10.4.31.111
Host is up (0.0088s latency).
Not shown: 993 filtered ports
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49154/tcp open  unknown
49155/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 4.85 seconds
```

(The `Stats:` line is wrapped here to fit; on screen it is one line.)

How to read it:

- `Stats` and `Timing` lines are progress reports. Nmap prints them on a slow scan or when you press a key. They are not results.
- `Host is up (0.0088s latency)` means a reply came back in 8.8 ms.
- `Not shown: 993 filtered ports` plus the 7 listed ports equals 1000, the default number of ports scanned. 993 ports gave no answer (firewall).
- 80, 135, 139, 445, 3389 are open. This pattern (135, 139, 445, 3389) means Windows.
- 49154 and 49155 show `unknown` because they are high dynamic ports with no standard name.
- `scanned in 4.85 seconds`: the whole scan time.

Command 2 (from the notes): discovery only, same host.

```
nmap -sn 10.4.31.111
```

Part	Meaning
-sn	ping scan: host discovery only, no port scan

Output (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 12:02 IST
Note: Host seems down. If it is really up, but blocking our ping probes,
try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
```

How to read it: the same host that had 5 open ports is reported down. The message is Nmap telling you the probes may have been blocked. This is the key lesson of the block.

Variations and notes:

- `-sn` was called `-sP` in old versions.
- `-Pn -sn` together skips discovery and port scan but still allows NSE host scripts (Nmap reference).
- `-Pn` on a big range scans every address and is slow. On `/24` that is 256 hosts.

Common mistakes:

- Trusting "0 hosts up" and moving on. Fix: retry with `-PS`, `-PA`, other ports, then `-Pn` (3.15 to 3.18).
- Using `-Pn` on a whole subnet by habit. Fix: use `-sn` first, then `-Pn` only on hosts you believe exist.

Worked example. Brief: host 10.4.31.111 should exist.

1. `nmap -sn 10.4.31.111` -> "Host seems down".
2. `nmap -Pn 10.4.31.111` -> 7 ports open, Windows pattern.
3. You write: "10.4.31.111 Windows, 80, 135, 139, 445, 3389, 49154, 49155. Blocks ping."
4. Next step: version scan (3.19).

Lab question angle. "Which ports are open on the host?" Even if the host looks down, use `-Pn`. "How many hosts are in the subnet?" Do not stop at the `-sn` count if a host is known to exist.

Why it works and how defenders stop it. Hosts reveal themselves by answering probes. Windows Firewall drops ICMP echo by default (verify), which is why the host looked down. A firewall that drops everything except needed ports is the correct defense.

Memory aid. `-sn` = "stop after the ping" (no port scan). `-Pn` = "Pretend it is alive" (no ping).

3.13 Ping Sweep with IP Packets: `--send-ip`

In one line: `--send-ip` makes Nmap send raw IP packets instead of Ethernet frames, so on a local network it does not use ARP for discovery.

Theory in simple words. Nmap can send packets at two levels. Raw Ethernet frames (layer 2) or raw IP packets (layer 3). When you are root and the targets sit on your own Ethernet network, Nmap uses ARP for host discovery unless `--send-ip` is given (Nmap reference). ARP is reliable but it only works locally. `--send-ip` makes Nmap behave as if the targets were remote, using IP-level probes (ICMP and TCP).

Command from the notes:

```
nmap -sn 10.10.22.0/24 --send-ip
```

Part	Meaning
-sn	discovery only
10.10.22.0/24	all 256 addresses, 10.10.22.0 to 10.10.22.255
--send-ip	send via raw IP sockets instead of Ethernet frames

The notes show Wireshark open with the title "Capturing from eth1" and a filter showing "No Packets" at that moment. The purpose is to watch the scan traffic on the interface while the sweep runs.

Illustrative output:

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 12:05 IST
Nmap scan report for 10.10.22.1
Host is up (0.00045s latency).
Nmap scan report for 10.10.22.5
Host is up (0.00061s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 5.20 seconds
```

How to read it: one "Nmap scan report" line per live host. The last line gives totals.

How to watch it (ADDED CONTEXT): start Wireshark on the interface that faces the target (`eth1` in the notes), then apply the display filter `arp || icmp || tcp`. Without `--send-ip` on a local subnet you see ARP "who has" requests. With it you see ICMP and TCP probes. The operating system may still send its own ARP requests to find MAC addresses (verify in the capture).

Common mistakes: forgetting `sudo`, then Nmap falls back to unprivileged probes. Capturing on the wrong interface and seeing nothing.

Worked example. A local host answers ARP but drops ICMP. `nmap -sn 10.10.22.0/24` finds it. `nmap -sn 10.10.22.0/24 --send-ip` may miss it. The difference tells you how a remote attacker would see that host.

Lab question angle. Mostly conceptual: "Which option forces Nmap to use IP packets instead of ARP?" -> `--send-ip`.

Why it works and how defenders stop it. IP-level probes depend on the target answering ICMP or TCP. Defenders drop them. They cannot drop ARP on the local segment.

Memory aid. `--send-ip` = "send like a remote scanner".

3.14 Scanning a Target List: `-iL`

In one line: `-iL` reads the targets from a file, one per line.

Theory in simple words. When you have many hosts from different ranges, typing them all is slow and error-prone. Put them in a text file and let Nmap read it.

Commands from the notes:

```
vim targets.txt
nmap -sn -iL targets.txt
```

Part	Meaning
vim targets.txt	open (create) the file in the vim editor
-sn	discovery only
-iL targets.txt	take targets from input list file `targets.txt`

The file content is not visible in the notes. A valid file looks like this (illustrative):

```
10.4.20.217
10.4.23.227
10.4.27.83
10.10.23.0/24
10.4.31.100-110
```

Rules: one target per line. Single IPs, CIDR ranges, and dash ranges all work.

Output (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:04 IST
Nmap done: 12 IP addresses (0 hosts up) scanned in 11.09 seconds
```

How to read it: Nmap read 12 addresses from the file. None answered the default probes. Zero hosts is a signal to try other probe types, not proof of empty space.

Useful variations:

- `nmap -sL -iL targets.txt` lists what Nmap would scan without sending probes. It confirms the file was parsed correctly.
- `nmap -iL targets.txt --exclude 10.4.27.83` removes one host.
- In vim: press `i` to type, `Esc`, then `:wq` and Enter to save and quit. Use `nano targets.txt` if you prefer.

Common mistakes: Windows line endings or extra spaces (rare), a missing file path (give the full path), forgetting the file must be in the current directory.

Worked example. You collected IPs from three pings. You write them to `targets.txt`, run `nmap -sn -iL targets.txt`, and later `nmap -sS -sV -iL targets.txt` to scan them all in one pass.

Lab question angle. "How many hosts did the scan cover?" -> read the "N IP addresses" total.

Why it works and how defenders stop it. This is a convenience option, not an attack. Nothing to defend except normal scan detection.

Memory aid. `-iL` = Input List.

3.15 TCP SYN Ping: `-PS`

In one line: `-PS` checks if a host is alive by sending a TCP SYN to a port and treating any answer as proof of life.

Theory in simple words. If ICMP is blocked, firewalls often still allow TCP to a service port. Nmap sends a SYN to a chosen port. A SYN/ACK (port open) or an RST (port closed) both prove a machine is there. Silence means the probe was dropped or the host is down. The default port is 80. This is the "half-open" technique, since the handshake is never completed.

The notes show a sequence against 10.4.23.227. All commands used `-sn`.

Command	Result	Time
`nmap -sn 10.4.23.227`	Host seems down	3.08 s
`nmap -sn -PS 10.4.23.227`	Host is up	0.10 s
`nmap -sn -PS22 10.4.23.227`	Host seems down	2.07 s
`nmap -sn -PS1-1000 10.4.23.227`	Host is up	17.53 s
`nmap -sn -PS3389 10.4.23.227`	Host is up	0.11 s
`nmap -sn -PS80,3389,445 10.4.23.227`	Host is up	0.09 s

```
nmap -sn -PS 10.4.23.227
nmap -sn -PS22 10.4.23.227
nmap -sn -PS1-1000 10.4.23.227
nmap -sn -PS3389 10.4.23.227
nmap -sn -PS80,3389,445 10.4.23.227
```

Part	Meaning
-PS	TCP SYN ping to the default port 80
-PS22	SYN ping to port 22
-PS1-1000	SYN ping to every port from 1 to 1000
-PS3389	SYN ping to port 3389 (RDP)
-PS80,3389,445	SYN ping to a list of three ports

Syntax rule: there is no space between `-PS` and the port list. Use commas for a list and a dash for a range.

Output (from the notes, the successful forms):

```
Nmap scan report for 10.4.23.227
Host is up (0.0083s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
```

How to read it: the plain ping scan said down. Port 22 gave no answer, so the firewall drops port 22 probes or nothing listens behind a drop rule. Ports 80, 3389, and 445 answered. The long range `1-1000` also worked but took 17.53 s and printed `Ping Scan Timing` progress lines (about 6 percent after 13 s). A short list of likely ports is faster and gives the same answer.

Common mistakes: putting a space (`-PS 22`), which makes Nmap read `22` as a target. Using a huge range when five ports would do.

Useful port list for Windows-like hosts: 80, 135, 139, 445, 3389. For Linux-like hosts: 21, 22, 25, 80, 443.

Worked example. `nmap -sn 10.4.23.0/24` finds nothing. You run `nmap -sn -PS80,443,445,3389,22 10.4.23.0/24` and now 10.4.23.227 appears. You follow with `nmap -Pn -sS -p- 10.4.23.227` (see 3.19).

Lab question angle. "Find the live host that does not answer ping." Use `-PS` with common ports, then `-Pn`.

Why it works and how defenders stop it. A server has to answer on its service ports. Defenders use stateful firewalls that drop SYN to everything except needed ports, and rate-limit SYNs.

Memory aid. `-PS` = "Probe with SYN". No space, ports glued on.

3.16 TCP ACK Ping: `-PA`

In one line: `-PA` sends a TCP ACK packet to a port, expecting an RST from a live host.

Theory in simple words. An ACK packet without a prior connection is "unexpected". A live host answers with RST. A stateless firewall may let it through. A stateful firewall (one that tracks connections) drops it because no SYN came first. The default port is 80.

Command from the notes:

```
nmap -sn -PA 10.4.23.227
```

Part	Meaning
-PA	TCP ACK ping to the default port 80
-sn	discovery only

Output (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:23 IST
Note: Host seems down. If it is really up, but blocking our ping probes,
try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.08 seconds
```

How to read it: the same host answered `-PS` (3.15) but not `-PA`. This tells you the firewall is stateful: it accepts new connections (SYN) to some ports but drops stray ACKs. That is information about the firewall, not about the host being down.

Variations: `-PA80,443`, `-PA3389`. As a non-root user, Nmap cannot send raw ACKs and uses a connect call instead (verify).

Worked example. `-PS` works, `-PA` fails -> stateful firewall. If both fail but `-Pn` finds open ports -> all pings are blocked and only service ports answer.

Lab question angle. Rare as a direct question. It supports questions such as "Which probe type reveals the host?"

Why it works and how defenders stop it. ACK probes pass stateless packet filters. Stateful firewalls drop them.

Memory aid. `-PA` = ACK from nowhere. Stateful firewalls say "no".

3.17 ICMP Echo Ping: `-PE`

In one line: `-PE` forces Nmap to use only a classic ICMP echo request (ping) for discovery.

Theory in simple words. This is the same message `ping` sends (Type 8, Code 0, see 3.11). Nmap normally sends echo plus other probes. With `-PE` and `-sn` you test just the ICMP echo.

Command from the notes (run twice in the screenshot):

```
nmap -sn -PE 10.4.23.227
```

Part	Meaning
-PE	ICMP echo request ping only
-sn	discovery only

Output (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:31 IST
Note: Host seems down. If it is really up, but blocking our ping probes,
try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.12 seconds
```

The second run printed the same message and took 2.07 s.

How to read it: no echo reply. Combined with 3.15 (SYN ping works), the host blocks ICMP echo but not TCP.

> **ADDED CONTEXT:** Related ping options: `-PP` ICMP timestamp, `-PM` ICMP netmask, `-PU` UDP ping, `-PR` ARP ping (local only). Combine them: `nmap -sn -PE -PS80,443 -PA80 10.4.23.0/24`.

Worked example. Create a test table of results for host X: `-PE` fail, `-PS` ok, `-PA` fail. Conclusion: ICMP blocked, stateful firewall, TCP service reachable.

Lab question angle. "Does the host respond to ICMP?" -> test with `-PE` or `ping`.

Why it works and how defenders stop it. Defenders block ICMP echo at the host or perimeter firewall.

Memory aid. `-PE` = Echo.

3.18 Verbose Ping Scans and Several Probes: `-v`, `-T4`

In one line: `-v` shows what Nmap is doing, `-T4` speeds it up, and a list of ports in `-PS` raises your chance of finding a host.

Theory in simple words. Verbosity means more output about the work in progress. Timing templates (`-T0` to `-T5`) set how aggressive the scan is. Both are covered fully in 3.29 and 3.31. Here they appear with host discovery.

Commands from the notes:

```
nmap -sn -v -T4 10.4.20.217
nmap -sn -PS21,22,25,80,445,3389,8080 -T4 10.4.20.217
```

Part	Meaning
-sn	discovery only
-v	verbose: print extra detail
-T4	timing template 4 (aggressive, faster)
-PS21,22,25,80,445,3389,8080	SYN ping to these 7 ports

Output of the first command (from the notes):

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:37 IST
Initiating Ping Scan at 15:37
Scanning 10.4.20.217 [4 ports]
Completed Ping Scan at 15:37, 2.05s elapsed (1 total hosts)
Nmap scan report for 10.4.20.217 [host down]
Read data files from: /usr/bin/../share/nmap
Note: Host seems down. If it is really up, but blocking our ping probes,
try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.08 seconds
           Raw packets sent: 8 (304B) | Rcvd: 0 (0B)
```

Output of the second command (from the notes):

```
Nmap scan report for 10.4.20.217
Host is up (0.0083s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

How to read it:

- `Scanning 10.4.20.217 [4 ports]`: the default probe set has 4 probes (ICMP echo, SYN to 443, ACK to 80, ICMP timestamp, see 3.12).
- `Raw packets sent: 8 (304B) | Rcvd: 0 (0B)`: 8 packets out, nothing back. Likely two tries of 4 probes (verify).
- `[host down]` in the report line shows only with `-v`.
- The second command finds the host in 0.08 s because one of the 7 chosen ports answered.
- The notes end with the second command typed again. No new output.

Which probe do I choose? (decision table)

Situation	Try
Same subnet as you	plain `-sn` (ARP)
Remote, default fails	`-sn -PS80,443,445,3389,22`
Still nothing	`-sn -PA80,443` and `-PE`

Situation	Try
You are sure the host exists	`-Pn`

Common mistakes: reading `-v` output as results (the result lines are the "scan report" lines). Using `-T5` on unstable labs and missing hosts.

Worked example. A host at 10.4.20.217 is in the scope sheet. `-sn -v -T4` says down. You send a 7-port SYN ping and it is up. You then run a port scan with `-Pn` as a safe habit (3.19).

Lab question angle. "Which hosts are up in the subnet?" Build the list with several probe types, and cross-check with `fping` (3.11).

Why it works and how defenders stop it. Same as 3.15 to 3.17. Defenders detect many fast probes with an IDS.

Memory aid. One probe misses, many probes hit. `-v` = Verbose, `-T4` = Turbo.

3.19 Full-Port, Service and OS Scan: `-sS -sV -O --osscan-guess -p-`

In one line: One command that finds every open TCP port on a host, the service and version behind each, and a guess of the operating system.

Theory in simple words. Port scanning means sending probes to ports and reading the answers. A port is in one of these states:

- **open:** a program listens and accepts connections.
- **closed:** the host answered (RST) but nothing listens.
- **filtered:** no answer or an ICMP error. A firewall hides the real state.
- **unfiltered:** reachable, but Nmap cannot tell open from closed (only from `-sA`, see 3.26).
- **open|filtered** and **closed|filtered:** Nmap cannot decide between the two. Seen mostly in UDP, FIN, Null, and Xmas scans.

Host fingerprinting has two parts. Service fingerprinting (`-sV`) connects to open ports and matches the replies against a database to name the service and version. OS fingerprinting (`-O`) sends special packets and compares how the TCP/IP stack reacts with a database of known systems. Analogy: knowing which doors are open is a port scan. Reading the name plate on each door is `-sV`. Guessing the building type from how the doors are built is `-O`.

Command from the notes (target 192.31.214.3, Nmap 7.70):

```
sudo nmap -T4 -sS -sV -O --osscan-guess -p- 192.31.214.3
```

Part	Meaning
sudo	run as root (needed for -sS and -O)
-T4	aggressive timing, faster (see 3.29)
-sS	TCP SYN scan ("stealth" or half-open scan)

Part	Meaning
-sV	detect service and version on open ports
-O	detect the operating system
--osscan-guess	guess OS more aggressively when no exact match
-p-	scan all ports, 1 to 65535
192.31.214.3	the target

The notes show only the start line of this run:

```
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 16:43 UTC
```

Illustrative full output:

```
Nmap scan report for 192.31.214.3
Host is up (0.00012s latency).
Not shown: 65532 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh     OpenSSH 7.6p1 (protocol 2.0)
80/tcp    open  http    Apache httpd 2.4.29
55413/tcp open  ftp     vsftpd 3.0.3
MAC Address: 02:42:C0:E0:4D:03 (Unknown)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.6
Network Distance: 1 hop
Service Info: OS: Unix

Nmap done: 1 IP address (1 host up) scanned in 28.40 seconds
```

How to read it:

- `Not shown: 65532 closed ports`: the ports you do not see are closed. Only 3 are interesting.
- `VERSION` is the answer for most exam questions that ask for a product and version.
- `Running`, `OS details`: the OS guess. With `--osscan-guess` you may see several guesses with percentages.
- `Network Distance: 1 hop`: the host is one router hop away (on the same segment it says 1 hop).
- `Service Info: OS: Unix`: a hint taken from the service banners.

How the SYN scan works:

1. Nmap sends SYN to a port.
2. SYN/ACK -> open. Nmap sends RST so the connection never completes (half-open).
3. RST -> closed.
4. No answer or ICMP unreachable -> filtered.

Variations:

- `-sT` (connect scan) if you are not root. It completes the handshake and is easier to log.

- `-p 1-1000`, `-p 22,80,443` for fewer ports.
- `--osscan-limit` (from the help screen) tries OS detection only on hosts that look promising: at least one open and one closed port.
- Two-stage method: `nmap -p- --min-rate 1000 -T4 TARGET` first, then `nmap -sV -sC -p 22,80,55413 TARGET` on only the open ports.

Common mistakes:

- Scanning only the default 1000 ports and missing services on 55413 or 41288.
- Running without root: `-sS` and `-O` fail with "requires root privileges".
- Trusting `-O` blindly. It is a guess. Check with banners and service info.
- Forgetting `-Pn` when the target blocks ping.

Worked example. Target 192.31.214.3.

1. `sudo nmap -T4 -sS -sV -O --osscan-guess -p- 192.31.214.3` returns three open ports.
2. Notes: "55413 vsftpd 3.0.3, Linux 4.x".
3. Next: check FTP for anonymous login (3.24), then browse port 80.

Lab question angle. "What version of the FTP server runs on the target?" -> `-sV` line. "What port does the web server listen on?" -> full-port scan, not the default. "What OS does the target run?" -> `-O` plus service info.

Why it works and how defenders stop it. TCP stacks answer in slightly different ways, and services announce themselves. Defenders hide banners, close ports, filter traffic, and use an IDS to alert on scans.

Memory aid. "S-S-V-O-P": SYN scan, Service, OS, all Ports. Full port first, details second.

3.20 Scan Techniques and Port Specification

In one line: Nmap has many TCP and UDP scan types and many ways to say which ports to scan. This block explains the options from the help screen.

Theory in simple words. Different scan types send different packets. Pick the one that fits your rights (root or not), the protocol (TCP or UDP), and the firewall. The help screen in the notes shows two groups: scan techniques and port specification. It also shows two options in the group before them: `--system-dns` and `--traceroute`.

Scan techniques (from the notes):

Option	Name	What it sends
-sS	SYN scan	SYN, never completes the handshake
-sT	Connect scan	Full handshake via the OS (no root)
-sA	ACK scan	ACK, used to map firewall rules (3.26)
-sW	Window scan	ACK, reads the TCP window of the RST
-sM	Maimon scan	FIN/ACK
-sU	UDP scan	UDP datagrams
-sN	Null scan	no flags set

Option	Name	What it sends
-sF	FIN scan	FIN flag only
-sX	Xmas scan	FIN, PSH, URG flags

More options from the same help section:

- `--scanflags <flags>`: build your own flag mix, for example `--scanflags URGACKPSHRSTSYNFIN`.
- `-sI <zombie host[:probeport]>`: idle scan. It bounces the scan off a "zombie" host with predictable IP IDs, so your IP never talks to the target. Advanced; know the name.
- `-sY` and `-sZ`: SCTP INIT and COOKIE-ECHO scans (SCTP is a rare transport protocol).
- `-sO`: IP protocol scan. It finds which IP protocols (such as 1, 6, 17) the host supports, not ports.
- `-b <FTP relay host>`: FTP bounce scan, which uses a vulnerable FTP server as a relay. Old and mostly patched.

> **ADDED CONTEXT:** Null, FIN, and Xmas scans follow the RFC rule: a closed port answers RST, an open port stays silent, so the result is `open|filtered`. Windows hosts answer RST either way, so these scans do not work on Windows. They can slip past some simple filters. For the eJPT, SYN, connect, UDP, and ACK scans matter most.

Port specification (from the notes):

- `-p <port ranges>`: scan only these ports. Examples: `-p22` (one port), `-p1-65535` (all ports), `-p U:53,111,137,T:21-25,80,139,8080,S:9` (U = UDP, T = TCP, S = SCTP).
- `--exclude-ports <port ranges>`: skip these ports.
- `-F`: fast mode, scans fewer ports than the default (the top 100).
- `-r`: scan ports in order, do not randomize.
- `--top-ports <number>`: scan the N most common ports.
- `--port-ratio <ratio>`: scan ports more common than the ratio (0 to 1).

Other options on the same screen:

- `--system-dns`: use the operating system's DNS resolver instead of Nmap's built-in one. Slower but follows your system settings.
- `--traceroute`: trace the hop path to each host. It shows routers between you and the target, which helps map networks behind routers.

Example commands (ADDED CONTEXT, each uses options from the notes):

```
sudo nmap -sU -p 53,161 10.10.10.5
nmap -sT -F 10.10.10.5
sudo nmap -sS --top-ports 20 --traceroute 10.10.10.5
```

Part	Meaning
-sU -p 53,161	UDP scan of ports 53 and 161
-sT -F	connect scan of the fast set (top 100 ports)

Part	Meaning
--top-ports 20	the 20 most common ports
--traceroute	add the hop path after the scan

Illustrative output of the UDP scan:

```
PORT    STATE         SERVICE
53/udp  open          domain
161/udp open|filtered snmp
```

How to read it: DNS answered, so it is open. SNMP gave no answer, so it may be open or filtered. UDP uses silence a lot, so confirm with `-sV` or a targeted script.

Illustrative traceroute lines:

```
TRACEROUTE (using port 80/tcp)
HOP RTT     ADDRESS
1   0.40 ms 10.10.10.1
2   0.80 ms 10.10.10.5
```

Common mistakes: a UDP scan of all ports (very slow, use `--top-ports` or named ports). Using `-sS` without root. Mixing `-p-` with `-F` (an explicit `-p` wins; verify).

Worked example. A host shows no TCP DNS. `sudo nmap -sU -p 53 10.10.10.5` shows open. You write "DNS on UDP only".

Lab question angle. "Which scan type does not need root?" -> connect scan `-sT`. "Which option scans the top 100 ports?" -> `-F`. "Which scan finds firewall rules?" -> `-sA`.

Why it works and how defenders stop it. Each scan type exploits how TCP, UDP, or IP must respond. Defenders use stateful firewalls and IDS that spot odd flag combinations.

Memory aid. S = SYN, T = Throughout (connect), U = UDP, A = ACK (firewall), N/F/X = flag tricks. `-p-` = all ports, `-F` = Fast.

3.21 Service Version Detection and Intensity

In one line: `-sV` names the service and its version, and `--version-intensity` sets how hard Nmap tries.

Theory in simple words. For each open port, Nmap sends probes from its service database and matches the replies. Intensity runs from 0 (light, few probes) to 9 (every probe). The default is 7 (verify with your version). More probes means better accuracy but a slower scan.

Command from the notes:

```
sudo nmap -T4 -sS -sV --version-intensity 0 -O --osscan-guess -p- \
  192.31.214.3
```

This is the command of 3.19 with one added option.

Part	Meaning
--version-intensity 0	lightest version detection: fewest probes

The notes show the command typed with no output yet. The result has the same layout as 3.19, but some services may show less detail.

Options from the help screen:

Option	Meaning
-sV	probe open ports to find service and version
--version-intensity N	level 0 (light) to 9 (try all probes)
--version-light	most likely probes only (intensity 2)
--version-all	every probe (intensity 9)
--version-trace	show detailed version scan activity (debug)

Useful variations: `-sV --version-all` when a service shows `unknown` or `tcpwrapped`. `-sV --version-trace` to see the probes and replies.

Common mistakes: using intensity 0 on a hard target and trusting "unknown". Running `-sV` on all 65535 ports with intensity 9 on a slow target.

Worked example. The first pass is `--version-intensity 0` to get a fast picture. One port shows `unknown`. You re-scan only that port: `sudo nmap -sV --version-all -p 41288 192.31.214.3`.

Lab question angle. "What is the exact version of the service?" Use `-sV`. If it is vague, raise the intensity or grab the banner with netcat.

Why it works and how defenders stop it. Services give away product names in banners and behavior. Defenders can change banners, but behavior still leaks.

Memory aid. Intensity is a dial: 0 quick glance, 9 full inspection, 7 default.

3.22 The Nmap Scripting Engine and Default Scripts: `-sC`

In one line: NSE (Nmap Scripting Engine) runs small Lua scripts that gather more information, and `-sC` runs the default set.

Theory in simple words. An NSE script is a ready-made check for one service or vulnerability. Examples: list MongoDB databases, test FTP anonymous login. Scripts are grouped in categories. `-sC` is a shortcut for `--script=default`.

Categories (ADDED CONTEXT): `auth`, `broadcast`, `brute`, `default`, `discovery`, `dos`, `exploit`, `external`, `fuzzer`, `intrusive`, `malware`, `safe`, `version`, `vuln`. In the notes, `mongodb-databases` is in "default discovery safe" and `memcached-info` is in "discovery safe" (so `-sC` does not run it, see 3.24).

Command from the notes (target 192.224.77.3):

```
sudo nmap -sS -sV -sC -p- -T4 192.224.77.3
```

Part	Meaning
-sS	SYN scan
-sV	version detection
-sC	run the default NSE scripts
-p-	all 65535 TCP ports
-T4	aggressive timing

The notes show the start line (2023-11-28 18:20 UTC). The screenshots that follow start in the middle of the output (the beginning is cut off):

```
|       pinned = 0
|       indexCounters
|         hits = 2
|         misses = 0
|         missRatio = 0.0
|         resets = 0
|_        accesses = 2
41288/tcp open  achat   AChat chat system
55413/tcp open  ftp     vsftpd 3.0.3
MAC Address: 02:42:C0:E0:4D:03 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.26 seconds
```

The tail of the same script output in another screenshot:

```
|       operation
|         fastmod = 0
|         scanAndOrder = 0
|         idhack = 0
|       version = 2.6.10
|_      process = mongod
```

How to read it:

- Lines starting with `|` and `|_` are script output, drawn as a tree under the port line. `|_` marks the last line.
- This is the `mongodb-info` output: the MongoDB version is `2.6.10` and the process is `mongod`.
- [UNCLEAR on page 8] The port line of the MongoDB service is cut off. MongoDB uses 27017 by default (see 3.7). Treat the exact port as unknown.
- `41288/tcp open achat AChat chat system`: a service that Nmap labels as the AChat chat system.
- `55413/tcp open ftp vsftpd 3.0.3`: FTP on a high port. The version is 3.0.3.
- `MAC Address: 02:42:C0:E0:4D:03 (Unknown)`: the target is on your local segment. The 02:42 prefix is typical for Docker containers (added, verify).
- `Service Info: OS: Unix`: OS hint from banners.

> **TRAP:** In a later screenshot (3.24), the same port 41288 on the same target is named `memcached Memcached`, not `achat`. The first name looks like a wrong guess or a restarted lab. Double check odd-port results by hand with a banner grab before you answer a question.

Common mistakes: expecting `-sC` to run every script. It runs only category `default`. Forgetting that default scripts touch the services (they are "safe" but not silent).

Worked example. `-sC` shows MongoDB with no login needed (version 2.6.10, counters). You write "MongoDB 2.6.10, open without authentication" and follow up with `mongodb-databases` (3.23).

Lab question angle. "What version of MongoDB runs on the target?" -> `2.6.10` from `mongodb-info`. "Is anonymous access allowed?" -> read the script block.

Why it works and how defenders stop it. Scripts talk to the service like a normal client. Defenders require authentication, bind databases to localhost, and firewall their ports.

Memory aid. `-sC` = "Scripts, Common ones". `|` lines = script says.

3.23 Finding Scripts and Reading Their Help

In one line: Scripts live in one folder. You find one by name with `ls` and `grep`, and you read what it does with `--script-help`.

Theory in simple words. NSE has hundreds of scripts. Their file names start with the service name, for example `mongodb-info.nse`. So to find scripts for a service, list the folder and filter by the service name.

Commands from the notes:

```
ls -al /usr/share/nmap/scripts/ | grep -e "mongodb"
nmap --script-help=mongodb-databases
ls -al /usr/share/nmap/scripts/ | grep -e "memcached"
nmap --script-help=memcached-info
```

Part	Meaning
ls -al	list files in long format, including hidden
/usr/share/nmap/scripts/	folder of NSE scripts on Kali
the pipe	pass the list to the next command
grep -e "mongodb"	keep only lines matching the pattern
--script-help=NAME	print category and description of a script

Output (from the notes):

```
-rw-r--r-- 1 root root 2578 Jan  9  2019 mongodb-brute.nse
-rw-r--r-- 1 root root 2583 Jan  9  2019 mongodb-databases.nse
-rw-r--r-- 1 root root 3663 Jan  9  2019 mongodb-info.nse
```

```
mongodb-databases
Categories: default discovery safe
https://nmap.org/nsedoc/scripts/mongodb-databases.html

  Attempts to get a list of tables from a MongoDB database.
```

```
-rw-r--r-- 1 root root 5655 Jan  9  2019 memcached-info.nse
```

```
memcached-info
Categories: discovery safe
https://nmap.org/nsedoc/scripts/memcached-info.html

  Retrieves information (including system architecture, process ID, and
  server time) from distributed memory object caching system memcached.
```

How to read it:

- Three MongoDB scripts exist: `mongodb-brute` (password guessing), `mongodb-databases` (list databases), `mongodb-info` (server info).
- `Categories` tells you the risk. `safe` scripts do not harm the target. `brute` scripts guess passwords.
- The URL points to the full documentation on nmap.org.
- Memcached has one script, `memcached-info`, which shows architecture, process ID, and server time.

Variations:

- `ls /usr/share/nmap/scripts | grep -i smb` finds SMB scripts (case-insensitive).
- `nmap --script-help "ftp-*"` shows help for a whole family (quote the star).
- `locate '*.nse' | grep http` also works if the locate database exists.

Common mistakes: using the wrong folder on another system (Kali uses `/usr/share/nmap/scripts/`; verify with `locate nmap.nse`). Typing the script name with `.nse` in `--script-help` (not needed).

Worked example. A scan shows port 6379 (Redis). You run `ls /usr/share/nmap/scripts | grep -i redis`, read `nmap --script-help=redis-info`, then run the script (3.24).

Lab question angle. "Which NSE script lists databases on MongoDB?" -> `mongodb-databases`. You find it with `ls | grep`.

Why it works and how defenders stop it. This is local research on your own machine. Nothing to defend.

Memory aid. Find with `ls | grep`, learn with `--script-help`, run with `--script`.

3.24 Running Chosen Scripts: `--script`

In one line: `--script=NAME` runs exactly the script (or family of scripts) you pick.

Theory in simple words. Use `-sC` for a broad pass. Use `--script` when a service needs a focused check. You can name one script, a comma list, a category, or a pattern with `*`.

Commands from the notes (target 192.224.77.3):

```
sudo nmap -sS -sV --script=memcached-info -p- -T4 192.224.77.3
sudo nmap -sS -sV --script=ftp-anon -p55413 -T4 192.224.77.3
sudo nmap -sS -sV "--script=ftp-*" -p55413 -T4 192.224.77.3
```

Part	Meaning
--script=memcached-info	run only this script
--script=ftp-anon	test for anonymous FTP login
--script=ftp-*	run all scripts whose name starts with ftp-
-p55413	scan only port 55413 (no space needed)
-p-	all ports (used in the memcached run)

The notes type the ftp wildcard without quotes. Quoting it is safer, because your shell may expand `ftp-*` into file names in the current folder.

Output of the memcached run: the notes show only the start line (2023-11-28 18:30 UTC). The earlier `-sV` run named port 41288 as `memcached Memcached`. Illustrative script output (field names vary, verify):

```
41288/tcp open  memcached Memcached
| memcached-info:
|   Process ID         1
|   Architecture       64 bit
|_  Server time        2023-11-28T18:31:04
```

Output of the ftp-anon run (from the notes):

```
Nmap scan report for target-1 (192.224.77.3)
Host is up (0.000036s latency).

PORT      STATE SERVICE VERSION
55413/tcp open  ftp     vsftpd 3.0.3
MAC Address: 02:42:C0:E0:4D:03 (Unknown)
Service Info: OS: Unix

Nmap done: 1 IP address (1 host up) scanned in 3.62 seconds
```

How to read it: `ftp-anon` printed nothing under the port line. When anonymous login works, the script adds `Anonymous FTP login allowed (FTP code 230)` and a file listing. No block means no anonymous access was reported. The scan took 3.62 s because only one port was checked.

Output of the ftp-* run (from the notes) shows only progress lines:

```
Stats: 0:00:02 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 83.64% done; ETC: 18:35 (0:00:00 remaining)
Stats: 0:00:04 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 81.67% done; ETC: 18:35 (0:00:01 remaining)
```

The notes end by running `ftp-anon` again.

How to read it: `Script Scan` means the scripts are still running. The percentage can go down, as the estimate changes. The `ftp-*` pattern includes scripts from other categories

such as `ftp-brute` (password guessing). Check what a family contains before you run it on a real target.

> **ADDED CONTEXT:** Other ftp scripts: `ftp-syst` (system info), `ftp-bounce`, `ftp-vsftpd-backdoor` (only relevant to vsftpd 2.3.4), `ftp-proftpd-backdoor`. The target runs vsftpd 3.0.3, which is not the backdoored 2.3.4 (verify the list with `ls | grep ftp`).

Useful variations:

- `--script=ftp-anon,ftp-syst` runs two scripts.
- `--script "default and safe"` combines categories.
- `--script vuln` runs the vulnerability checks. Use with care (intrusive scripts can disturb services).

Common mistakes: running `ftp-*` and forgetting that brute-force scripts are included. Not reading the script output tree and missing a finding.

Worked example. Goal: check FTP on 55413.

1. `-sV` shows vsftpd 3.0.3 (3.19).
2. `--script=ftp-anon -p55413` shows no anonymous login.
3. You note: "FTP anon: not allowed. Next: credentials from other services".

Lab question angle. "Does the FTP server allow anonymous login?" -> `ftp-anon` result, or `ftp 192.224.77.3 55413` test, logging in as `anonymous` (added). "What is the server time of the memcached service?" -> `memcached-info`.

Why it works and how defenders stop it. Misconfigured services allow anonymous or unauthenticated access. Defenders disable anonymous FTP and require authentication on every service.

Memory aid. One service, one script family: `ftp-*`, `smb-*`, `http-*`. Always quote the star.

3.25 Firewall Evasion: Packet Fragmentation (`-f`, `--mtu`)

In one line: `-f` and `--mtu` cut each scan packet into tiny IP fragments so simple firewalls and IDS may not see the full header.

Theory in simple words. A firewall is a filter that allows or blocks traffic by rules (IP, port, flags). An IDS (intrusion detection system) watches traffic and raises alerts. Some of them only inspect whole packets. If the TCP header is split across several small fragments, a weak device may not match its rules. Analogy: a guard who checks complete letters but lets torn pieces through, and the pieces are glued together inside.

How Nmap does it (verified in the Nmap reference):

- `-f` splits the packet into pieces of 8 bytes or less after the IP header. A 20-byte TCP header becomes three fragments (8, 8, 4 bytes). Use `-f` twice (`-ff`) for 16-byte pieces.
- `--mtu N` sets your own fragment size. N must be a multiple of 8. The reference says not to use `-f` together with `--mtu`. The notes use both.

- It works only for raw-packet features: TCP and UDP port scans (not connect scan) and OS detection. Version detection and NSE rely on the OS TCP stack and are not fragmented, even if you add `-f`.
- Fragments may be rebuilt by the OS or by a defending device. Check in Wireshark.

Commands from the notes (Nmap 7.91, target 10.4.27.83):

```
sudo nmap -Pn -sS -sV -p80,445,3389 -f --mtu 32 10.4.27.83
sudo nmap -Pn -sS -sV -p80,445,3389 -f --mtu 4 10.4.27.83
sudo nmap -Pn -sS -sV -p80,445,3389 -f --mtu 8 10.4.27.83
```

Part	Meaning
-Pn	skip host discovery
-sS -sV	SYN scan plus version detection
-p80,445,3389	scan only these three ports
-f	fragment the packets
--mtu 32	fragment size of 32 bytes (a multiple of 8)
--mtu 4	invalid: not a multiple of 8
--mtu 8	the smallest valid size

Output of the first command (from the notes; two long lines are wrapped):

```
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan
times will be slower.
Starting Nmap 7.91 ( https://nmap.org ) at 2023-11-29 00:42 IST
Nmap scan report for 10.4.27.83
Host is up (0.011s latency).

PORT      STATE  SERVICE      VERSION
80/tcp    closed http
445/tcp   open   microsoft-ds Microsoft Windows Server 2008 R2 - 2012
                              microsoft-ds
3389/tcp  open   ssl/ms-wbt-server?
Service Info: OS: Windows Server 2008 R2 - 2012; CPE:
cpe:/o:microsoft:windows

Nmap done: 1 IP address (1 host up) scanned in 66.16 seconds
```

Output of the second command (from the notes):

```
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan
times will be slower.
Data payload MTU must be >0 and multiple of 8

QUITTING!
```

The third command only shows its start line (2023-11-29 00:44 IST).

How to read it:

- The fragmented scan still gets the right answer: 80 closed, 445 open (SMB on Windows Server 2008 R2 to 2012), 3389 open (RDP).

- `ssl/ms-wbt-server?` the `?` means Nmap is not fully sure, and `ssl/` means the service speaks TLS.
- 66.16 seconds for three ports is slow. Fragmentation and `-Pn` cost time.
- `--mtu 4` stops the scan at once. The rule: the value must be above 0 and a multiple of 8.

Common mistakes: an MTU that is not a multiple of 8. Expecting `-sV` traffic to be fragmented. Assuming evasion always works (modern IDS reassemble fragments).

Worked example. Port scan results differ between a normal and a fragmented scan? Probably a device in the middle treats fragments differently. You note which ports changed, since it tells you the firewall behavior.

Lab question angle. "What is the OS of the host at 10.4.27.83?" -> Windows Server 2008 R2 to 2012 from `-sV`. "Why did Nmap quit with --mtu 4?" -> must be a multiple of 8.

Why it works and how defenders stop it. Weak filters inspect each fragment alone. Defenders turn on fragment reassembly (virtual reassembly) or drop tiny fragments.

Memory aid. Fragment sizes come in 8s: 8, 16, 24, 32.

3.26 ACK Scan to Map Firewall Rules: `-sA`

In one line: An ACK scan does not find open ports. It shows which ports a firewall lets packets reach (unfiltered) and which it blocks (filtered).

Theory in simple words. Nmap sends a TCP packet with only the ACK flag. A host that receives it replies with RST whether the port is open or closed. So Nmap cannot say open or closed, but it can say "the packet got through" or "something blocked it".

- RST received -> `unfiltered` (the firewall lets this traffic reach the host).
- No reply or an ICMP error -> `filtered` (a firewall drops or rejects it).

Analogy: you cannot see inside the rooms, but you learn which hallways have locked gates.

Command from the notes:

```
sudo nmap -Pn -sA -p445,3389 10.4.27.83
```

Part	Meaning
-Pn	skip host discovery
-sA	ACK scan
-p445,3389	check only SMB and RDP

Output (from the notes). The screenshot starts in the middle of an earlier scan, shown first. Its command is cut off [UNCLEAR on page 8].

```
Host is up (0.0096s latency).
Not shown: 92 closed ports
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49152/tcp open  unknown
49153/tcp open  unknown
49154/tcp open  unknown
49155/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
```

The 8 listed ports plus 92 closed make 100, so that earlier scan most likely used `-F` (top 100 ports; my inference, verify). Now the ACK scan:

```
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan
times will be slower.
Starting Nmap 7.91 ( https://nmap.org ) at 2023-11-29 00:33 IST
Nmap scan report for 10.4.27.83
Host is up (0.0093s latency).

PORT     STATE      SERVICE
445/tcp  filtered   microsoft-ds
3389/tcp unfiltered ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds
```

How to read it:

- The earlier scan found 445 open. The ACK scan calls it `filtered`. One reading: the firewall allows new connections (SYN) to 445 but drops stray ACK packets, so it tracks connection state (stateful).
- 3389 is `unfiltered`: the ACK packet reached the host and an RST came back. The firewall does not block this kind of packet to 3389.
- The scan finished in 0.14 s. Few ports, fast result.

Useful variations: `-sA -p-` to map all ports. Combine with `-sS` results: ports `open` in `-sS` and `filtered` in `-sA` show the firewall tracks state.

Common mistakes: reading `unfiltered` as `open`. It is not.

Worked example. A SYN scan shows 445 open and 3389 open. An ACK scan shows 445 filtered and 3389 unfiltered. You write: "Firewall treats 445 and 3389 differently. Stateful rule on 445."

Lab question angle. "Which port is filtered by the firewall?" -> run `-sA` and read the state column. "What does unfiltered mean?" -> reachable, state unknown.

Why it works and how defenders stop it. Stateless filters pass ACK packets. Stateful firewalls drop ACK packets that do not belong to a known connection.

Memory aid. ACK scan = firewall X-ray. It answers "blocked or not", never "open or closed".

3.27 Payload Length and Decoys (`--data-length`, `-D`)

In one line: `--data-length` pads packets with random bytes. `-D` hides your IP among fake source addresses.

Theory in simple words.

- **Custom payload length:** Nmap normally sends packets with no data after the headers. Some IDS rules match these exact small sizes. `--data-length N` appends N random bytes, so the packets look different.
- **Decoys:** `-D` sends the same probes from several source addresses, mixed with your real one. The target's logs show many scanners. The defender cannot tell which one is real. You still get your answers because replies to your real address reach you.

Analogy: ten people knock on a door at once and only one of them is you.

Command from the notes (target 10.4.27.83):

```
sudo nmap -Pn -sS -sV -p445,3389 -f --data-length 200 \
  -D 10.10.23.1,10.10.23.2 10.4.27.83
```

Part	Meaning
-Pn -sS -sV	no discovery, SYN scan, version detection
-p445,3389	only these two ports
-f	fragment packets (3.25)
--data-length 200	add 200 random bytes to each packet
-D 10.10.23.1,10.10.23.2	use these two IPs as decoys
10.4.27.83	the real target

The notes show only the start of the run (2023-11-29 00:52 IST, Nmap 7.91). Illustrative result:

```
PORT     STATE SERVICE      VERSION
445/tcp  open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012
3389/tcp open  ms-wbt-server
```

How to read it: the result looks the same as a normal scan. The difference is on the target side. Its logs show probes from 10.10.23.1, 10.10.23.2, and your real IP.

Rules for decoys (Nmap reference):

- Decoys should be live hosts. If they are down, your probes can look like a SYN flood, and the target may block everyone.
- Put `ME` in the list (`-D decoy1,ME,decoy2`) to choose where your real IP sits. Otherwise Nmap picks a random place.
- `-D RND:5` makes Nmap invent 5 random decoys.
- Decoys work in the host discovery and port scan phases and in OS detection (`-O`). They do not work with version detection or a connect scan. So the `-sV` phase still uses your real IP.

- The decoy IPs in the notes (10.10.23.x) are in a different network than the target (10.4.27.x). Real decoys should usually be near the target.

Common mistakes: using dead decoys. Thinking decoys hide you from every log.

Worked example. You scan with `-D RND:5,ME` and the target's firewall log shows six source addresses. The defender cannot tell which one is you.

Lab question angle. Mostly conceptual: "Which Nmap option hides the scanner among fake sources?" -> `-D`. "Which option changes the packet size?" -> `--data-length`.

Why it works and how defenders stop it. The target sees only source IPs, and you can send any source IP in a raw packet. Defenders correlate TTL values and routing paths, use ingress filtering, and block scan patterns whatever the source.

Memory aid. `-D` = Decoy. `--data-length` = pad the packet.

3.28 Other Spoofing Options: `-S`, `-e`, `-g`, `--proxies`, `--data`

In one line: These options from the help screen fake the source address, pick the network interface, fix the source port, relay through proxies, or add your own payload.

Theory in simple words. The notes list them in the "Firewall/IDS evasion and spoofing" section. They were not run in the video, so the examples here are ADDED CONTEXT.

Option	Meaning
-S IP	spoof the source address
-e iface	use this network interface
-g PORT or --source-port PORT	send from this source port
--proxies url1,url2	relay connections through HTTP or SOCKS4 proxies
--data HEX	append a custom binary payload

Details:

- **`-S`:** replies go to the spoofed address, not to you. It is useful only when you can see those replies (same network, or sniffing). It usually needs `-e` and `-Pn` too (Nmap reference).
- **`-e eth1`:** needed when Nmap picks the wrong interface, or with `-S`. Matches the `eth1` capture in 3.13.
- **`-g 53`:** some old firewalls trust traffic whose source port is 53 (DNS) or 20 (FTP data). Sending from that port may pass the filter.
- **`--proxies`:** experimental in Nmap (verify). It works for connect-style scanning only.
- **`--data 0xdeadbeef`:** appends the hex bytes you give. `--data-string "text"` appends text (verify).

Example commands (illustrative):

```
sudo nmap -Pn -sS -g 53 -p 80,445 10.4.27.83
sudo nmap -Pn -e eth1 -S 10.10.23.50 10.4.27.83
sudo nmap -Pn -sS -p 80 --data 0xdeadbeef 10.4.27.83
```

Part	Meaning
-g 53	source port 53 on every probe
-e eth1 -S 10.10.23.50	send from eth1 with a fake source IP
--data 0xdeadbeef	add four hex bytes to each packet

Illustrative output of the first command:

```
PORT    STATE  SERVICE
80/tcp  closed http
445/tcp open   microsoft-ds
```

How to read it: if a port is `filtered` in a normal scan and `open` with `-g 53`, the firewall has a source-port rule you can pass. The spoofed-source command prints nothing useful, because the answers go to 10.10.23.50.

Common mistakes: using `-S` and expecting results. Forgetting `-e` with `-S`.

Worked example. Port 445 is filtered. You retry with `-g 53`. If it turns open, you note the weak rule for the report.

Lab question angle. Name-the-option questions: "Which option sets the source port?" -> `-g` or `--source-port`.

Why it works and how defenders stop it. Old rules trust source ports and source addresses. Defenders use stateful rules and ingress filtering.

Memory aid. `-S` Spoof source, `-e` Egress interface, `-g` Give a source port.

3.29 Scan Timing and Performance

In one line: Timing options set how fast Nmap sends probes. Faster saves time, slower hides from the IDS and avoids dropped packets.

Theory in simple words. Time values are in seconds. Add `ms`, `s`, `m`, or `h` for milliseconds, seconds, minutes, or hours (for example `30m`). The simplest control is the timing template `-T<0-5>`, higher is faster.

Template	Name	Use
-T0	paranoid	very slow, IDS evasion
-T1	sneaky	slow, IDS evasion
-T2	polite	uses less bandwidth
-T3	normal	the default
-T4	aggressive	fast, good for labs and good networks
-T5	insane	fastest, may miss results

Options from the help screen:

- `--min-hostgroup` and `--max-hostgroup <size>`: how many hosts are scanned in parallel.
- `--min-parallelism` and `--max-parallelism <numprobes>`: probes in flight at once.

- `--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout <time>`: how long to wait for one reply.
- `--max-retries <tries>`: how many times to resend a probe.
- `--host-timeout <time>`: give up on a host after this long (see 3.30).
- `--scan-delay` and `--max-scan-delay <time>`: pause between probes.
- `--min-rate <number>`: send at least this many packets per second.
- `--max-rate <number>`: send at most this many packets per second.

Example commands (ADDED CONTEXT):

```
sudo nmap -p- --min-rate 1000 -T4 10.10.10.5
sudo nmap -sS -p 1-1000 --scan-delay 1s --max-retries 1 10.10.10.5
```

Part	Meaning
--min-rate 1000	at least 1000 packets per second
--scan-delay 1s	wait 1 second between probes
--max-retries 1	resend an unanswered probe once

Illustrative result: the first command finishes a 65535-port scan in under a minute on a lab network. The second takes minutes but is quiet.

Common mistakes: `-T5` or a huge `--min-rate` on a shaky lab link, which drops replies and hides open ports. Rerun the missed ports with `-T3` if a result looks odd.

Worked example. `-p- -T4` takes 4 minutes. `--min-rate 1000` takes 40 seconds and finds the same three ports. You then run `-sV -sC` on those three only.

Lab question angle. Speed is a tool for you: use `-T4` as the default in the exam lab, and slow down only when ports disappear between runs.

Why it works and how defenders stop it. Fast scans are loud. Defenders set rate limits and thresholds in the IDS.

Memory aid. T0 and T1 are for hiding, T3 is default, T4 is the lab choice, T5 loses data.

3.30 Scanning a Subnet and `--host-timeout`

In one line: `--host-timeout` stops Nmap from spending too long on one slow host in a big scan.

Theory in simple words. On a /24 range with `-sV`, one stubborn host can hold the scan for minutes. `--host-timeout 5s` tells Nmap to give up on any host after 5 seconds. The cost: slow hosts may be dropped with incomplete data.

The notes show the end of a scan of 10.10.23.0/24 with progress lines (output from the notes, shortened):

```
Stats: 0:00:38 elapsed; 255 hosts completed (2 up), 1 undergoing Script Scan
NSE Timing: About 98.84% done; ETC: 01:11 (0:00:00 remaining)
Nmap scan report for attackdefense.com (10.10.23.2)
Host is up (0.0000050s latency).

Not shown: 98 closed ports

PORT      STATE SERVICE       VERSION
3389/tcp  open  ms-wbt-server xrdp
10000/tcp open  http          Apache Tomcat 8.5.57

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 256 IP addresses (2 hosts up) scanned in 45.63 seconds
```

The next command from the notes:

```
sudo nmap -sS -sV -F --host-timeout 5s 10.10.23.0/24
```

Part	Meaning
-sS -sV	SYN scan and version detection
-F	fast mode, top 100 ports
--host-timeout 5s	abandon any host after 5 seconds
10.10.23.0/24	256 addresses of the lab subnet

The notes show only its start line (Nmap 7.91, 2023-11-29 01:11 IST).

How to read the first output:

- `255 hosts completed (2 up), 1 undergoing Script Scan`: 256 addresses in the range. Two are up, and one host is still running scripts. The earlier command therefore included scripts such as `-sC` (the command is cut off, [UNCLEAR on page 9]).
- `98 closed ports` plus the 2 listed make 100, matching `-F`.
- `10.10.23.2` runs xrdp (an RDP server for Linux) on 3389 and Apache Tomcat 8.5.57 on port 10000 (Tomcat's default is 8080, so port 10000 is a custom choice).
- The second up host is not visible in the screenshot [UNCLEAR on page 9].
- `256 IP addresses (2 hosts up) scanned in 45.63 seconds`: a full /24 in under a minute.

Common mistakes: setting the host timeout too low with `-sV` and `-p-`. Hosts are cut off and ports are missing. Use it only for quick sweeps, with `-F`. Verify any skipped host with a longer, targeted scan.

Worked example. After the quick sweep you have two live hosts. You scan each in full: `sudo nmap -sS -sV -p- 10.10.23.2`.

Lab question angle. "How many hosts are up in the subnet?" -> the `(N hosts up)` total. "Which version of Tomcat runs?" -> `8.5.57`, on port 10000.

Why it works and how defenders stop it. Hosts that hang or rate-limit probes slow scans. Defenders use tarpits and rate limits on purpose.

Memory aid. Host-timeout is a kitchen timer: ring after 5 seconds and move to the next host.

3.31 Nmap Output and Verbosity

In one line: Verbosity controls how much Nmap prints while it works, and output options save results to files.

Theory in simple words. The course overview lists "Nmap Output & Verbosity". The notes show `-v` (3.18) and the `Stats:` progress lines (3.12, 3.24, 3.30). The rest is ADDED CONTEXT because exam notes live and die by saved output.

Option	Meaning
-v	more detail while scanning. `-vv` for even more
-oN file	save normal text output
-oX file	save XML output
-oG file	save grepable output
-oA base	save all three formats with base name
--open	show only open ports
--reason	show why each port got its state
--stats-every 10s	print a progress line every 10 seconds

Example (ADDED CONTEXT):

```
sudo nmap -sS -sV -p- -T4 --open -oA scan_10.10.10.5 10.10.10.5
ls scan_10.10.10.5.*
```

Part	Meaning
--open	print only ports that are open
-oA scan_10.10.10.5	write three files with this base name
ls scan_10.10.10.5.*	list the files created

```
scan_10.10.10.5.gnmap  scan_10.10.10.5.nmap  scan_10.10.10.5.xml
```

How to read it: `.nmap` is the readable text, `.gnmap` is easy to search with `grep`, `.xml` is for tools such as Metasploit's `db_import`. During a long scan, press Enter to get a progress line like those in the notes.

Common mistakes: not saving output and losing a 10-minute scan. Reusing a file name overwrites it.

Worked example. Save every scan with `-oA`, named by host. When a question asks "which version?", search `grep -i version *.nmap` instead of rescanning.

Lab question angle. The exam lab does not ask for files. Saved scans protect your answers, and let you answer in any order.

Why it works and how defenders stop it. Not an attack. Defenders log scans too.

AI angle (course content, unsure if examined). The 2026 course "Offensive AI: Generative AI for Pentesters" covers LLMs and prompt engineering. A simple use: paste a scan with fake or lab data and ask an LLM to summarize it. Example prompt:

"Summarize this Nmap output as a table of port, service, version, and the next enumeration step." Never paste real client data into public AI tools, and verify every claim yourself.

Memory aid. `-oA` = Output All. Always save.

4. Chained Scenarios

All IPs are fake lab addresses. Full explanations are in the blocks named in brackets.

Scenario A: The host that ignores ping

Goal: find and profile a Windows host that blocks ICMP. Scope: 10.10.10.0/24, attacker at 10.10.10.2.

```
ip -br a
```

Confirm my own IP and subnet (see 3.4).

```
fping -a -g 10.10.10.0/24 2>/dev/null
```

Fast ICMP sweep. Finds only hosts that answer ping (see 3.11).

```
sudo nmap -sn 10.10.10.0/24
```

Nmap default discovery (ARP on the local segment). Cross-check (see 3.12).

```
sudo nmap -sn -PS80,135,445,3389,22 10.10.10.0/24
```

SYN pings reveal hosts that block ICMP (see 3.15).

```
sudo nmap -Pn -sS -p- --min-rate 1000 -T4 10.10.10.7
```

Fast all-port scan of the new host, with discovery off (see 3.19, 3.29).

```
sudo nmap -Pn -sS -sV -sC -O -p135,139,445,3389 -oA host7 10.10.10.7
```

Versions, default scripts, and OS on the open ports only. Saved for notes (see 3.19, 3.22, 3.31).

Notes to write: host 10.10.10.7, Windows pattern (135, 139, 445, 3389), blocks ICMP, SMB version string, OS guess. Next module topic: SMB enumeration.

Scenario B: Services on odd ports

Goal: map a Linux host that hides services on high ports. Target 192.168.50.20.

```
sudo nmap -sS -p- -T4 --open 192.168.50.20
```

Default 1000 ports would miss 41288 and 55413 (see 3.7, 3.19).

```
sudo nmap -sS -sV -p22,41288,55413 192.168.50.20
```

Names the odd-port services (see 3.19, 3.21).

```
ls /usr/share/nmap/scripts/ | grep -i -e ftp -e memcached
```

Find scripts that match the services found (see 3.23).

```
sudo nmap -sV --script=ftp-anon -p55413 192.168.50.20
```

Check anonymous FTP login (see 3.24).

```
sudo nmap -sV --script=memcached-info -p41288 192.168.50.20
```

Pull memcached details. If the port was named wrongly in the first scan, this shows it (see 3.22, 3.24).

Notes to write: FTP version, anonymous result, memcached details, any disagreement between two scans of the same port.

Scenario C: Firewall in the way, then the next subnet

Goal: learn what the firewall does, then map a second subnet from a compromised host. Target 10.4.27.83.

```
sudo nmap -Pn -sS -p80,445,3389 10.4.27.83
```

Baseline SYN scan: which ports answer (see 3.19).

```
sudo nmap -Pn -sA -p80,445,3389 10.4.27.83
```

ACK scan: which ports the firewall blocks (see 3.26).

```
sudo nmap -Pn -sS -f --mtu 16 -g 53 -p445 10.4.27.83
```

Retry a filtered port with fragments and source port 53 (see 3.25, 3.28).

```
sudo nmap -Pn -sS -sV -p445,3389 -D RND:5 10.4.27.83
```

Noisy decoys when the lab says to be quiet (see 3.27).

After you gain a shell on the host and find a second interface with `ip a` (see 3.4):

```
fping -a -g 172.16.5.0/24 2>/dev/null
```

Sweep the hidden subnet from the compromised host (see 3.11). Port scans through a pivot are the topic of a later module.

Notes to write: filtered versus open ports, which evasion changed the result, the hidden subnet range, live hosts in it.

5. Decision Guide and Exam Strategy

5.1 If I see X, I try Y

If I see	I try
"Host seems down" but the host must exist	`-Pn`, then `-PS` on 80, 443, 445, 3389, 22
Target is on my own subnet	plain `-sn` (ARP)
Target is behind a router	`-sn -PS ports` or `-Pn`
Only ports 80, 135, 139, 445, 3389 open	Windows: SMB and RDP next
22 open and 80 open	Linux web server: SSH later, web first
A port number I do not know	`-sV -p PORT`, then banner grab
Service shows `unknown` or `tcpwrapped`	`--version-all`, then netcat
`filtered` ports everywhere	`-sA`, then `-f`, `-g 53`, `-D`
UDP service expected (DNS, SNMP)	`-sU -p 53,161`
Scan too slow	`-T4`, `--min-rate`, `--top-ports`, `-F`
Ports vanish between runs	slow down with `-T3` or `--max-retries 3`
A service name (ftp, smb)	list the scripts folder and filter by name

5.2 Enumeration checklist for this module

1. Write down scope, my IP, and my subnet.
2. Host discovery: `fping`, `nmap -sn`, then SYN pings on common ports.
3. For each live host: all-port scan (`-p-`), save with `-oA`.
4. On open ports only: `-sV -sC`, and `-O`.
5. For each service: look for scripts with `ls | grep`, run safe ones.
6. Re-scan odd or `unknown` ports with `--version-all`.
7. Check UDP on 53, 69, 123, 161.
8. Record firewall behavior (`-sA`).
9. Check `ip a` on every compromised host for new subnets.
10. Update the notes table after every scan.

5.3 Common traps

> **TRAP:** Trusting "Host seems down". Many labs block ping on purpose.
>
> **TRAP:** Scanning only the default 1000 ports. The notes show services on 41288, 55413, and 10000.
>
> **TRAP:** Using `-Pn` on a whole /24 and waiting forever. Find live hosts first.
>
> **TRAP:** Trusting a service name from one scan. Port 41288 was named two different ways in the notes (3.22).
>
> **TRAP:** Putting a space after `-PS` or `-p`. Keep ports attached: `-PS22`, `-p445`.
>
> **TRAP:** Reading `unfiltered` (ACK scan) as `open`. It only means reachable.
>
> **TRAP:** Forgetting `sudo`. SYN scans, OS detection, and UDP scans need root.
>
> **TRAP:** Counting your own IP or the router as targets.

5.4 How to take notes in the lab

Keep one table per host, updated after every scan:

```
Host         OS guess   Port   Service  Version       Notes
10.10.10.7   Windows    445    smb      Win 2008 R2   anon? not tested
10.10.10.7   Windows    3389   rdp      -             needs creds
```

Also keep: the exact command that produced each fact, the time, and a "to do" list (services not yet enumerated). Save every scan with `-oA`. Take a screenshot of key results.

5.5 Turning findings into answers

- **Multiple choice:** run the check, do not guess. Eliminate options by evidence. If two options look right, find the detail that separates them (version number, port, state).
- **Dynamic value:** the answer must come from your own lab instance (a port, a flag, a version). Copy it exactly from the output. Do not retype from memory. Watch case and spaces.
- **Fill-in-the-blank:** use the shortest exact form the tool prints (for example `vsftpd 3.0.3`, `2.6.10`, `55413`). If the question asks for a service, give the service name, not the product.
- **Time:** if a question needs a result you cannot get, mark it and move on. Return after enumeration of other hosts, since later findings (credentials, new subnets) often unlock earlier questions.
- **Check twice:** re-read the question after you have the answer. Did it ask for TCP or UDP? Which host? Which port?

6. Practice

All tasks use fake addresses. Do them in a lab you own or in an authorized training lab.

6.1 Lab-Style Tasks

Task 1: Find your subnet. Scenario: you start in a new lab and need the range to scan. Do: list your interfaces and routes.

```
ip -br a
ip route
```

Expect:

```
eth0     UP        10.10.10.2/24
10.10.10.0/24 dev eth0 proto kernel scope link src 10.10.10.2
```

How to get the answer: the address with its mask on the lab interface gives the subnet. Here 10.10.10.0/24. (Block 3.4)

Task 2: Who listens on port 4444? Scenario: a Python program runs on your machine. Do: list TCP sockets with program names.

```
sudo netstat -antp | grep 4444
```

Expect:

```
tcp6   0   0 :::4444     :::*     LISTEN      825/python3
```

How to get the answer: read the last column: PID 825, program python3. Without sudo the column shows a dash. (Block 3.9)

Task 3: Count live hosts with a ping sweep. Scenario: subnet 10.10.23.0/24. Do: sweep and count.

```
fping -a -g 10.10.23.0/24 2>/dev/null | wc -l
```

Expect:

```
2
```

How to get the answer: `wc -l` counts the lines, one per live host. `-a` prints only live hosts. (Block 3.11)

Task 4: The host that "is down". Scenario: 10.4.31.111 must exist but a ping scan says down. Do: compare discovery with and without it.

```
sudo nmap -sn 10.4.31.111
sudo nmap -Pn 10.4.31.111
```

Expect: the first prints "Host seems down", the second lists open ports (80, 135, 139, 445, 3389 in the notes). How to get the answer: `-Pn` skips discovery, so Nmap scans the ports anyway. (Block 3.12)

Task 5: Prove what packets Nmap sends. Scenario: you want to see ARP versus IP discovery on a local subnet. Do: start Wireshark on the lab interface (filter `arp || icmp`), then run both sweeps.

```
sudo nmap -sn 10.10.22.0/24
sudo nmap -sn 10.10.22.0/24 --send-ip
```

Expect: the first run shows ARP requests. The second shows ICMP and TCP probes from Nmap (the OS may still send some ARP, verify). How to get the answer: compare the Protocol column in Wireshark. (Block 3.13)

Task 6: Scan from a target file. Scenario: three hosts from different ranges. Do: build the list, check it, then scan it.

```
printf '10.4.20.217\n10.4.23.227\n10.4.27.83\n' > targets.txt
nmap -sL -iL targets.txt
sudo nmap -sn -iL targets.txt
```

Expect:

```
Nmap done: 3 IP addresses (0 hosts up) scanned in 0.05 seconds
Nmap done: 3 IP addresses (0 hosts up) scanned in 10.40 seconds
```

How to get the answer: the first line proves the file was read. If the second shows 0 hosts up, retry with `-PS` (Task 7). (Block 3.14)

Task 7: Find the host that answers only on RDP. Scenario: 10.4.23.227 does not answer ping or port 22. Do: try several SYN ping choices.

```
sudo nmap -sn -PS22 10.4.23.227
sudo nmap -sn -PS3389 10.4.23.227
sudo nmap -sn -PS80,3389,445 10.4.23.227
```

Expect: down, up, up (as in the notes). How to get the answer: the host replies on 3389. Note "RDP reachable, 22 filtered". (Block 3.15)

Task 8: Work out the firewall type from three probes. Scenario: the same host gives these results: `-PE` down, `-PA` down, `-PS80` up. Do: write the conclusion.

Expect: ICMP is blocked, a stateful firewall drops stray ACKs, TCP SYN to port 80 is allowed. How to get the answer: apply blocks 3.15, 3.16, and 3.17 in a table of probe versus result.

Task 9: Read a verbose discovery. Scenario: output says `Scanning 10.4.20.217 [4 ports]` and `Raw packets sent: 8 (304B) | Rcvd: 0 (0B)`. Do: explain it.

```
sudo nmap -sn -v -T4 10.4.20.217
```

Expect: the default 4 probes were sent, nothing came back, host marked down. How to get the answer: no replies does not mean no host. Run a 7-port SYN ping next. (Block 3.18)

Task 10: Find the FTP service on an odd port. Scenario: the web page mentions an FTP server. A default scan shows no port 21. Do: scan all ports with versions.

```
sudo nmap -sS -sV -p- -T4 192.224.77.3
```

Expect:

```
55413/tcp open  ftp     vsftpd 3.0.3
```

How to get the answer: the SERVICE and VERSION columns give "ftp" and "vsftpd 3.0.3" on 55413. (Blocks 3.7, 3.19)

Task 11: Guess the operating system. Scenario: target 192.31.214.3. Do: run OS detection with aggressive guessing.

```
sudo nmap -sS -O --osscan-guess -p- 192.31.214.3
```

Expect: lines `Running:` and `OS details:` with one or more guesses. How to get the answer: copy the OS family and version range. Cross-check with `Service Info: OS`. (Block 3.19)

Task 12: Compare light and full version detection. Scenario: one port shows `unknown`. Do: scan that port at two intensities.

```
sudo nmap -sV --version-intensity 0 -p 41288 192.224.77.3
sudo nmap -sV --version-all -p 41288 192.224.77.3
```

Expect: the second run names the service more reliably. How to get the answer: use the more detailed result, then confirm with a banner grab. (Block 3.21)

Task 13: Find the MongoDB version with default scripts. Scenario: target 192.224.77.3 runs a database. Do: run default scripts with version detection on all ports.

```
sudo nmap -sS -sV -sC -p- -T4 192.224.77.3
```

Expect:

```
|       version = 2.6.10
|_      process = mongod
```

How to get the answer: find the `mongodb-info` block and read `version`. (Block 3.22)

Task 14: Find and read a script. Scenario: a memcached service was found. Do: find the script file and read its help.

```
ls -al /usr/share/nmap/scripts/ | grep -e "memcached"
nmap --script-help=memcached-info
```

Expect: `memcached-info.nse` and the category line `Categories: discovery safe`. How to get the answer: the name from `ls`, the category from `--script-help`. Note `-sC` would not run it. (Block 3.23)

Task 15: Test anonymous FTP. Scenario: FTP on 55413. Do: run the single script on that port.

```
sudo nmap -sV --script=ftp-anon -p55413 192.224.77.3
```

Expect: the plain `vsftpd 3.0.3` line, no anonymous block. How to get the answer: no `Anonymous FTP login allowed` line means it was not reported. Confirm by hand: `ftp 192.224.77.3 55413`, user `anonymous`. (Block 3.24)

Task 16: Fragmentation rules. Scenario: your scan quits with `Data payload MTU must be >0 and multiple of 8`. Do: fix the command.

```
sudo nmap -Pn -sS -p445 -f --mtu 8 10.4.27.83
```

Expect: the scan runs. Sizes 8, 16, 24, and 32 are valid. How to get the answer: the value after `--mtu` must be a multiple of 8. (Block 3.25)

Task 17: Map the firewall with an ACK scan. Scenario: SYN scan shows 445 and 3389 open on 10.4.27.83. Do: run an ACK scan on the same ports.

```
sudo nmap -Pn -sA -p445,3389 10.4.27.83
```

Expect:

```
445/tcp  filtered   microsoft-ds
3389/tcp unfiltered ms-wbt-server
```

How to get the answer: `filtered` means the firewall blocks ACK packets to that port, `unfiltered` means they get through. State this in your notes. (Block 3.26)

Task 18: Use decoys safely. Scenario: the lab rules ask you to test IDS alerting on the target 10.4.27.83. Do: add two live hosts from the range as decoys and keep yourself in the list.

```
sudo nmap -Pn -sS -p445,3389 -D 10.4.27.10,ME,10.4.27.11 10.4.27.83
```

Expect: normal port results. The target's log shows three source addresses. How to get the answer: `ME` marks your position, the decoys should be live hosts. (Block 3.27)

Task 19: Compare timing templates. Scenario: an all-port scan takes too long. Do: time two runs.

```
time sudo nmap -p- -T3 10.10.10.5
time sudo nmap -p- -T4 --min-rate 1000 10.10.10.5
```

Expect: the second run is several times faster and finds the same open ports. How to get the answer: if the lists differ, rerun the difference at `-T3`. (Block 3.29)

Task 20: Quick subnet sweep with a host timeout. Scenario: 10.10.23.0/24 and little time. Do: fast scan with a 5 second limit per host.

```
sudo nmap -sS -sV -F --host-timeout 5s 10.10.23.0/24
```

Expect: a report for each live host and `256 IP addresses (2 hosts up)`. How to get the answer: read the totals line. Then scan each host fully (`-p-`). (Block 3.30)

Task 21: UDP DNS check. Scenario: TCP scan shows no DNS. Do: scan UDP port 53.

```
sudo nmap -sU -p 53 10.10.10.5
```

Expect:

```
53/udp open  domain
```

How to get the answer: `open` means DNS answers over UDP. `open|filtered` means unknown, so add `-sV`. (Blocks 3.8, 3.20)

Task 22: Read an IP header value. Scenario: a Wireshark capture of a ping reply shows `Protocol: ICMP (1)` and `Time to Live: 128`. Do: say what protocol number, and the likely OS.

How to get the answer: Protocol 1 is ICMP (6 is TCP, 17 is UDP). TTL 128 suggests Windows, assuming no router in between (ADDED CONTEXT). (Blocks 3.5, 3.11)

6.2 Exam-Style Questions

Q1 (multiple choice). Which Nmap option runs host discovery only, with no port scan? A) `-Pn` B) `-sn` C) `-sV` D) `-F`

Q2 (multiple choice). A host does not answer ping, but you know it exists. What is the best next step? A) Give up and mark it down B) Run `-sn` again C) Run `-Pn` or a SYN ping on common ports D) Use `-F`

Q3 (multiple choice). In the second step of the TCP three-way handshake, the server sends: A) SYN B) ACK C) SYN and ACK D) FIN

Q4 (fill-in). The IP protocol number for TCP is ____.

Q5 (fill-in). The ICMP echo reply has type ____.

Q6 (find the value). You scan 10.10.10.5 with `-sS -sV -p-` and see `55413/tcp open ftp vsftpd 3.0.3`. Give the port and the version string.

Q7 (multiple choice). `nmap --mtu 12 TARGET` fails. Why? A) Fragmentation needs root B) The MTU must be a multiple of 8 C) The MTU must be above 100 D) `--mtu` needs `-D`

Q8 (multiple choice). In an ACK scan, a port reported as `unfiltered` means: A) It is open B) It is closed C) It is reachable, open or closed is unknown D) It is blocked

Q9 (find the value). An `mongodb-info` output shows `version = 2.6.10` and `process = mongod`. What is the MongoDB version?

Q10 (fill-in). The folder that holds NSE scripts on Kali is ____.

6.3 Answer Key and Explanations

Q	Answer	Why
1	B	`-sn` is a ping scan: discovery only (3.12)
2	C	`-Pn` skips discovery, SYN pings use other ports (3.12, 3.15)
3	C	The server sends SYN/ACK (3.6)
4	6	ICMP is 1, TCP 6, UDP 17 (3.5)

Q	Answer	Why
5	0	Echo request is 8, echo reply is 0 (3.11)
6	55413 and vsftpd 3.0.3	PORT and VERSION columns (3.19)
7	B	The value must be above 0 and a multiple of 8 (3.25)
8	C	RST came back, so the packet passed the firewall (3.26)
9	2.6.10	Read the `version` field (3.22)
10	/usr/share/nmap/scripts/	Search it with `ls` and `grep` (3.23)

Other options explained: in Q1, `-Pn` skips discovery (the opposite), `-sV` is version detection, `-F` is fast mode. In Q2, option D reduces ports and does not help find the host. In Q7, root is not what stops the scan, the error text names the multiple-of-8 rule.

Tasks 1 to 22 answers: given inside each task under "How to get the answer".

7. Recap Card and Flashcards

7.1 Recap Card

One line per command. The trailing comment gives the section with the full explanation.

```
ip -br a                                        # 3.4
ip route                                        # 3.4
netstat -antp                                   # 3.9
fping -a -g 10.10.23.0/24                       # 3.11
fping -a -g 10.10.23.0/24 2>/dev/null           # 3.11
nmap -Pn 10.4.31.111                            # 3.12
nmap -sn 10.4.31.111                            # 3.12
nmap -sn 10.10.22.0/24 --send-ip                # 3.13
nmap -sn -iL targets.txt                        # 3.14
nmap -sn -PS 10.4.23.227                        # 3.15
nmap -sn -PS22 10.4.23.227                      # 3.15
nmap -sn -PS1-1000 10.4.23.227                  # 3.15
nmap -sn -PS80,3389,445 10.4.23.227             # 3.15
nmap -sn -PA 10.4.23.227                        # 3.16
nmap -sn -PE 10.4.23.227                        # 3.17
nmap -sn -v -T4 10.4.20.217                     # 3.18
nmap -sn -PS21,22,25,80,445,3389,8080 -T4 IP    # 3.18
nmap -T4 -sS -sV -O --osscan-guess -p- IP       # 3.19
nmap -sU -p 53,161 IP                           # 3.20
nmap -sS --top-ports 20 --traceroute IP         # 3.20
nmap -T4 -sS -sV --version-intensity 0 -p- IP   # 3.21
nmap -sS -sV -sC -p- -T4 IP                     # 3.22
ls -al /usr/share/nmap/scripts/ | grep -e NAME  # 3.23
nmap --script-help=NAME                         # 3.23
nmap -sS -sV --script=memcached-info -p- IP     # 3.24
nmap -sS -sV --script=ftp-anon -p55413 IP       # 3.24
nmap -sS -sV "--script=ftp-*" -p55413 IP        # 3.24
nmap -Pn -sS -sV -p80,445,3389 -f --mtu 32 IP   # 3.25
nmap -Pn -sA -p445,3389 IP                      # 3.26
nmap -Pn -sS -sV -p445,3389 -f --data-length 200 -D D1,D2 IP   # 3.27
nmap -Pn -sS -g 53 -p 80,445 IP                 # 3.28
nmap -p- --min-rate 1000 -T4 IP                 # 3.29
nmap -sS -sV -F --host-timeout 5s 10.10.23.0/24 # 3.30
nmap -sS -sV -p- -T4 --open -oA NAME IP         # 3.31
```

Quick facts: ICMP echo request 8, reply 0. IP protocol numbers 1 ICMP, 6 TCP, 17 UDP. Max port 65535. Default scan: top 1000 TCP ports. Scripts: `/usr/share/nmap/scripts/`. `--mtu` must be a multiple of 8.

7.2 Flashcards

- Q: What does `-sn` do? | A: Host discovery only, no port scan (3.12)
- Q: What does `-Pn` do? | A: Skips host discovery and scans ports anyway (3.12)
- Q: What does "Host seems down" really mean? | A: Probes got no answer. Try -Pn or other probes (3.12)
- Q: Which option reads targets from a file? | A: -iL file (3.14)
- Q: Which option sends a TCP SYN ping? | A: -PS, with an optional port list and no space (3.15)
- Q: Which option sends a TCP ACK ping? | A: -PA (3.16)

- Q: Which option sends an ICMP echo ping? | A: -PE (3.17)
- Q: Which option forces raw IP packets instead of ARP? | A: --send-ip (3.13)
- Q: What does `fping -a -g` do? | A: Ping sweep of a range, printing only live hosts (3.11)
- Q: ICMP type for echo request and reply? | A: 8 and 0 (3.11)
- Q: What are the 7 OSI layers from 7 down to 1? | A: Application, Presentation, Session, Transport, Network, Data Link, Physical (3.3)
- Q: Which layer do IP and ICMP work at? | A: Layer 3, the Network layer (3.4)
- Q: Which layer do TCP and UDP work at? | A: Layer 4, the Transport layer (3.6)
- Q: How long are IPv4 and IPv6 addresses? | A: 32 bits and 128 bits (3.4)
- Q: What is the minimum IPv4 header size? | A: 20 bytes (IHL 5) (3.5)
- Q: What do IP protocol numbers 1, 6, 17 stand for? | A: ICMP, TCP, UDP (3.5)
- Q: What does TTL do? | A: Counts down one per hop; at 0 the packet is dropped (3.5)
- Q: What are the three handshake steps? | A: SYN, SYN/ACK, ACK (3.6)
- Q: What is the highest port number? | A: 65535 (3.7)
- Q: What are registered ports? | A: 1024 to 49151 (3.7)
- Q: Which is connectionless, TCP or UDP? | A: UDP (3.8)
- Q: What does `netstat -antp` show? | A: All TCP sockets, numeric, with program names (3.9)
- Q: What reply does a closed port give to a SYN? | A: RST (3.19)
- Q: Which scan option needs no root? | A: -sT, the connect scan (3.20)
- Q: What does `-p-` scan? | A: All 65535 ports (3.19)
- Q: What does `-F` do? | A: Scans the top 100 ports (3.20)
- Q: What does `-sV` do? | A: Detects service and version on open ports (3.19)
- Q: What does `--version-intensity` take? | A: 0 (light) to 9 (all probes) (3.21)
- Q: What does `-sC` run? | A: The default NSE scripts (3.22)
- Q: Where are NSE scripts stored? | A: /usr/share/nmap/scripts/ (3.23)
- Q: How do I read what a script does? | A: nmap --script-help=NAME (3.23)
- Q: Why quote `--script "ftp-*"`? | A: The shell may expand the star (3.24)
- Q: Rule for `--mtu`? | A: Multiple of 8 (3.25)
- Q: What does `-sA` tell you? | A: Filtered or unfiltered, not open or closed (3.26)
- Q: What does `-D` do? | A: Adds decoy source IPs to hide the real scanner (3.27)
- Q: What does `--data-length 200` do? | A: Adds 200 random bytes to each packet (3.27)
- Q: Which option sets the source port? | A: -g or --source-port (3.28)
- Q: What are the -T levels? | A: 0 paranoid to 5 insane; 3 is default (3.29)
- Q: What does `--host-timeout 5s` do? | A: Skips a host after 5 seconds (3.30)
- Q: Which option saves all output formats? | A: -oA basename (3.31)

8. Glossary

- **ACK:** TCP flag that acknowledges received data.
- **ACK scan (-sA):** Nmap scan that maps firewall rules, not open ports.
- **Active information gathering:** Collecting data by talking to the target directly.
- **ARP:** Address Resolution Protocol. Finds the MAC address for an IP on the local network.
- **Broadcast:** One-to-all communication within a subnet.
- **CIDR:** Classless Inter-Domain Routing. Writes a range as address/prefix, like 10.10.10.0/24.
- **Closed port:** Reachable host, nothing listening. It answers RST.
- **Connect scan (-sT):** Scan using the full TCP handshake. No root needed.
- **Data Link layer:** OSI layer 2. Framing and MAC addressing.
- **Decoy (-D):** A fake source IP mixed into scan traffic.
- **DF:** Don't Fragment flag in the IP header.
- **DHCP:** Dynamic Host Configuration Protocol. Gives IP addresses automatically.
- **DSCP:** Differentiated Services Code Point. Priority bits in the ToS field.
- **ECN:** Explicit Congestion Notification. Congestion bits in the ToS field.
- **Echo request and reply:** ICMP types 8 and 0. The messages behind ping.
- **Filtered port:** No answer or an error. A firewall hides the real state.
- **Firewall:** A device or software that allows or blocks traffic by rules.
- **FIN:** TCP flag that ends a connection.
- **fping:** A tool that pings many hosts in parallel.
- **Fragmentation:** Splitting an IP packet into smaller pieces.
- **Header:** Control data at the front of a packet.
- **Host discovery:** Finding which IP addresses belong to live machines.
- **ICMP:** Internet Control Message Protocol. Errors and diagnostics, ping.
- **IDS:** Intrusion Detection System. Alerts on suspicious traffic.
- **IHL:** Internet Header Length. IP header size in 32-bit words.
- **IP:** Internet Protocol. Logical addressing and routing at layer 3.
- **IPv4 and IPv6:** IP versions with 32-bit and 128-bit addresses.
- **MF:** More Fragments flag in the IP header.
- **MTU:** Maximum Transmission Unit. The largest packet a link carries.
- **NSE:** Nmap Scripting Engine. Runs Lua scripts for extra checks.
- **Netstat:** Tool that lists sockets and listening ports on the local host.
- **Network layer:** OSI layer 3. Logical addressing and routing.
- **Nmap:** Network Mapper. The standard scanner.
- **Null, FIN, Xmas scans:** TCP scans using unusual flag sets.
- **Open port:** A program listens and accepts connections.
- **OS fingerprinting (-O):** Guessing the operating system from packet behavior.

- **OSI model:** Seven-layer reference model of network communication.
- **OSINT:** Open Source Intelligence. Public information about a target.
- **Packet:** A unit of data with a header and a payload.
- **Passive information gathering:** Collecting data without touching the target.
- **Payload:** The actual data inside a packet.
- **Ping sweep:** Sending ICMP echo requests to many addresses.
- **Port:** A 16-bit number that selects a service on a host.
- **Port scanning:** Probing ports to find which are open.
- **Protocol:** A set of rules for communication.
- **RST:** TCP flag that resets a connection. A closed port answers with it.
- **SYN:** TCP flag that starts a connection.
- **SYN scan (-sS):** Half-open scan. Sends SYN, never completes the handshake.
- **Service fingerprinting (-sV):** Naming the service and version on an open port.
- **Subnet:** A smaller network carved out of a larger one.
- **TCP:** Transmission Control Protocol. Reliable, connection-oriented, layer 4.
- **Three-way handshake:** SYN, SYN/ACK, ACK. How TCP opens a connection.
- **TTL:** Time-to-Live. Hop limit in the IP header.
- **ToS:** Type of Service. Priority field in the IP header.
- **Transport layer:** OSI layer 4. End-to-end delivery.
- **UDP:** User Datagram Protocol. Connectionless, no delivery guarantee.
- **Unfiltered:** From an ACK scan: reachable, open or closed unknown.
- **Wireshark:** Packet capture and analysis tool.

9. Coverage Report

9.1 Page to Section Map

Source page	Content	Guide section
1	Overview, objectives, methodology, active recon	2, 3.1
2	Scanning tree	3.1
2	Protocols, packets	3.2
3	OSI model	3.3
3	Network layer, IP, IPv4/IPv6, functions	3.4
4	IP functions, ICMP, DHCP	3.4
4	IPv4 header fields and layout	3.5
5	Reserved IPv4 addresses	3.4
5	Transport layer, TCP, handshake, header, flags	3.6
5	TCP port ranges and examples	3.7
6	TCP versus UDP	3.8
6	netstat	3.9
6	Host discovery techniques	3.10
6	Ping sweeps, ICMP types, fping	3.11
6	-Pn versus -sn	3.12
6	--send-ip	3.13
7	-iL	3.14
7	-PS	3.15
7	-PA	3.16
7	-PE	3.17
7	-sn -v -T4, multi-port SYN ping	3.18
7	Full-port, version, OS scan	3.19
7	--version-intensity 0	3.21
8	-sC, mongodb-info output	3.22
8	Script search and --script-help	3.23
8	memcached-info, ftp-anon, ftp-*	3.24
8	Help: scan techniques, ports, version	3.20, 3.21
8	-f, --mtu	3.25
8	-sA	3.26
8	--data-length, -D	3.27
9	Help: timing	3.29
9	Help: evasion and spoofing, osscan options	3.19, 3.27, 3.28
9	Subnet scan, --host-timeout	3.30

Source page	Content	Guide section
Overview	Output and verbosity	3.18, 3.31

9.2 [UNCLEAR] Items

- [UNCLEAR on page 8] The MongoDB port line is cut off in the `-sC` output. Default MongoDB port 27017 is only a guess.
- [UNCLEAR on page 8] The command of the scan that produced "Not shown: 92 closed ports" is cut off. `-F` is an inference from the 100-port total.
- [UNCLEAR on page 9] The command of the first subnet scan is cut off. The mention of Script Scan suggests `-sC` or similar. The second live host is not shown.
- [UNCLEAR on page 7] The contents of `targets.txt` (12 addresses) are not visible.
- No output is shown for the commands in 3.11 (fping), 3.19, 3.21, 3.24 (memcached), 3.25 (--mtu 8), 3.27. The guide shows illustrative output and labels it.

9.3 Items to Double Check

- Port 41288 on 192.224.77.3 is named `achat AChat chat system` in one screenshot and `memcached Memcached` in another. Likely a different run or restart. Verify by hand.
- The OSI table in the notes lists SSH on layers 6 and 7 and IMAP on layer 6. Many references put IMAP on layer 7.
- RFC 5735 is cited in the notes. I believe a newer RFC 6890 replaced it (verify).
- The notes use both `-f` and `--mtu`. The Nmap reference says not to combine them.
- The ACK scan result (445 filtered versus open in the SYN scan) is explained as a stateful firewall. This is an interpretation.
- Commands in the notes run as `root@attackdefense`. On your own Kali box use `sudo`.
- The notes use Nmap 7.70 and 7.91. Output formats can differ in your lab version.
- Exam facts used (45 questions, domain weights, 70 percent pass with domain minimums) come from your brief and were not conflicting with the notes.

9.4 Confirmation

Every line of `inventory.txt` (pages 1 to 9 and the overview-only topic) is placed in exactly one concept block, as shown in 9.1. Items labelled ADDED CONTEXT come from outside the notes.